S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Prelab

Aim: Introduction to Data Science tools (Python, Jupyter, Anaconda) and lab setup.

Name:

USN:

Semester / Year: Academic

Session: Date of

Performance: Date of

Submission:

❖ **Aim:** Installation of Linux Operating System.

❖ **Objectives:**

1. Understand the system requirements and compatibility for Linux OS installation.
2. Learn the step-by-step process to install and configure a Linux distribution.
3. Verify the installation and explore basic Linux commands for system setup.

❖ **Requirements:**

1. A bootable USB drive or DVD with the desired Linux distribution (e.g., Ubuntu, Fedora, Debian).
2. A computer system with minimum hardware requirements: 2 GB RAM, 20 GB free disk space, and a compatible processor.
3. Internet connection (optional, for updates during installation).
4. Software for creating a bootable USB, like Rufus or Etcher (if needed).
5. Basic knowledge of BIOS/UEFI settings for boot sequence configuration.

*****IN THIS PRACTICAL WE'LL BE INSTALLING UBUNTU*****

❖ **Prerequisite:**

Linux is an open-source operating system widely used for personal, professional, and server environments. Its flexibility, security, and community-driven development make it a popular choice for users. The installation of a Linux OS involves creating a bootable medium, setting up the system to boot from the medium, and following the installation wizard to partition the disk and configure system settings. Common distributions like Ubuntu, Fedora, and Debian offer user-friendly interfaces for easy installation. The process may include creating swap space, selecting a file system like ext4, and setting up user accounts. Post-installation tasks involve updating the system, installing necessary drivers, and customizing the environment. Understanding the installation process ensures better control over system performance and resource allocation, making Linux a powerful tool for both beginners and advanced users.

Steps to Make a Pendrive Bootable Using Rufus:

1. **Download and Open Rufus**: Download Rufus from its official website, install it, and launch the application.
2. **Insert Pendrive and Select ISO**: Connect the USB pendrive to your system. In Rufus, select your pendrive under "Device" and click "SELECT" to choose the Linux ISO file.
3. **Set Partition Scheme and File System**: Choose "GPT" for UEFI or "MBR" for BIOS under "Partition scheme," and ensure the file system is set to "FAT32."
4. **Start the Process**: Click "START," confirm the warning about data deletion, and wait for Rufus to create the bootable USB. Once done, eject the pendrive safely.

❖ **Theory:**

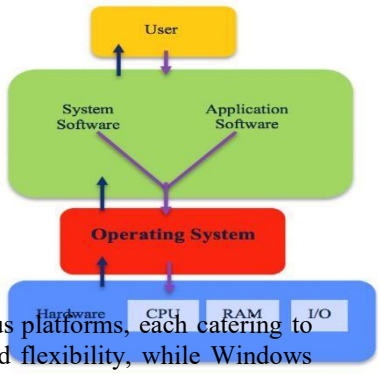
An operating system (OS) is system software that acts as an interface between computer hardware and the user. It manages hardware resources and provides essential services for executing application software. The OS is a vital component of a computer system, enabling users to interact with the system in a structured and efficient manner.

Functions of an Operating System:

1. **Process Management**: Handles the execution of multiple processes, ensuring smooth multitasking and optimal CPU usage.
2. **Memory Management**: Allocates and manages system memory, ensuring that each program gets the required memory without interfering with others.
3. **File System Management**: Organizes and manages data storage, allowing users to store, retrieve, and manipulate files.
4. **Device Management**: Coordinates and controls input/output devices such as keyboards, printers, and disk drives.
5. **Security and Access Control**: Protects data and system integrity by implementing user authentication and access restrictions.

Types of Operating Systems:

1. **Batch OS**: Executes tasks in batches without direct user interaction.
2. **Time-Sharing OS**: Allows multiple users to share system resources simultaneously.
3. **Distributed OS**: Manages a group of independent computers and makes them appear as a single system.
4. **Real-Time OS**: Designed for time-critical tasks where responses are needed within strict deadlines.
5. **Mobile OS**: Specialized for mobile devices, such as Android and iOS.



Operating systems like Windows, Linux, and macOS are widely used across various platforms, each catering to specific user needs. Linux, for example, is known for its open-source nature and flexibility, while Windows provides a user-friendly interface for personal and professional use.

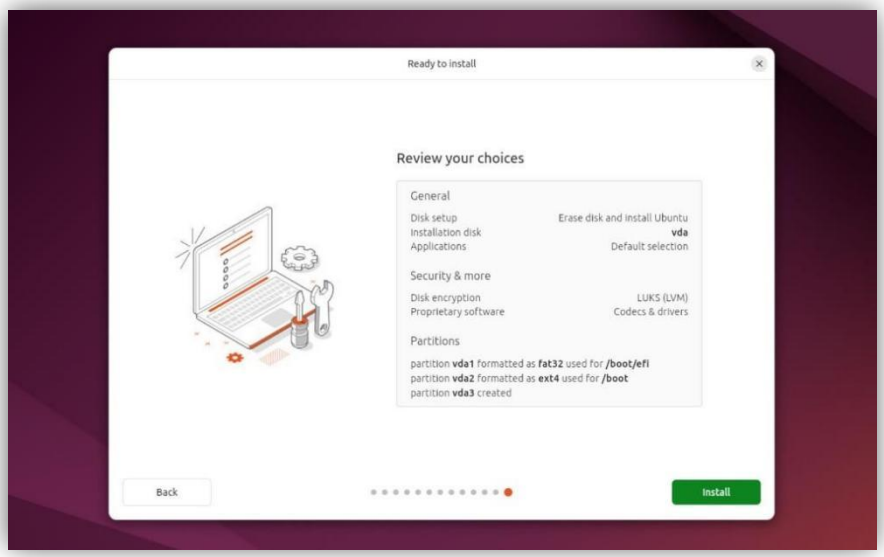
In summary, an operating system is the backbone of computer functionality, ensuring efficient resource management, user interaction, and system reliability. It continues to evolve, adapting to technological advancements and diverse user requirements.

❖ **Steps to Install Linux Operating System:**

1. **Prepare Bootable Media**: Use a tool like Rufus to create a bootable USB drive or DVD with the Linux distribution ISO.

2. **Configure BIOS/UEFI Settings**: Restart your computer and access the BIOS/UEFI settings (usually by pressing a key like F2, F10, or DEL during boot). Set the boot priority to USB or DVD.

3. **Boot from Media**: Insert the bootable USB or DVD and restart the computer. The system will boot into the Linux installer.
4. **Choose Installation Option**: Select "Install Linux" or a similar option from the menu. Some distributions may allow you to try the OS before installation.
5. **Partition the Disk**:
   - Select the partition scheme (automatic or manual).
   - Create required partitions (e.g., root `/`, swap, and optionally `/home`).
6. **Set Up User Details**: Enter your username, password, and system name.
7. **Select Time Zone**: Choose your location to configure the correct time and date settings.
8. **Begin Installation**: Review the settings and click "Install." The process will take a few minutes to complete.



9. **Remove Bootable Media**: Once installation is finished, remove the USB or DVD when prompted and restart the system.

10. **Post-Installation Configuration**: Log in to your new Linux system, update packages, and install additional software if needed.

    Commands to update:

Command	Use
sudo apt update	Fetches the latest information about available packages and versions.
sudo apt upgrade	Installs the latest versions of all currently installed packages.
sudo apt full-upgrade	Upgrades packages, adding or removing dependencies as required.
sudo apt autoremove	Removes unnecessary packages no longer needed as dependencies.
sudo reboot	Restarts the system to apply critical updates if required.

- **Conclusion:** The installation of the Linux operating system provides a hands-on understanding of system setup and configuration. It enables efficient utilization of resources and customization to suit user needs. Mastering this process builds a solid foundation for exploring advanced system administration tasks.

- **Discussion Questions:**

1. **Q: What is an operating system, and why is it important?**
   A: An operating system is system software that acts as an interface between hardware and users, managing resources like memory, processes, and files. It is essential for running applications and ensuring the computer operates efficiently.
2. **Q: What is the purpose of creating a bootable USB, and how is it done?**
   A: A bootable USB allows the installation of an operating system or running it directly without installation. It is created by writing an OS ISO image to a USB drive using tools like Rufus, ensuring the USB is formatted and properly configured for the target system's boot method.
3. **Q: Can you explain the difference between apt update and apt upgrade in Ubuntu?**
   A: apt update fetches the latest information about available packages from repositories but does not install updates. apt upgrade installs the latest versions of the currently installed packages based on the updated package list.
4. **Q: Why is partitioning necessary during OS installation, and what are the common partitions used?**
   A: Partitioning divides the storage into sections for efficient management and ensures proper allocation for the OS and user data. Common partitions include root (/), swap (for virtual memory), and optionally /home (for user files) and /boot (for bootloader files).
5. **Q: What steps should you follow after successfully installing a Linux OS?**
   A: Post-installation steps include updating the system using commands like sudo apt update and sudo apt upgrade, installing necessary drivers and software, customizing the desktop environment, and ensuring system security by configuring firewalls and user permissions.

- **References**:

https://ubuntu.com/tutorials/install-ubuntu-desktop#1-overview
*https://youtu.be/wjbbl0TTMeo?si=32l6h8VbcmU-euD_*
https://answers.microsoft.com/
https://rufus.ie/en/

Date:_____/_____/2026

Signature
Course Coordinator
B.Tech CSE(DS) Sem: 4
/ 2025-26

S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 02

Aim: To understand and demonstrate the use of basic commands in different operating systems (Windows, Linux, and UNIX) for managing files, directories, permissions, and user interactions through a terminal or command-line interface.

Name:

USN:

Semester / Year: Academic

Session: Date of

Performance: Date of

Submission:

- **Aim:** To understand and demonstrate the use of basic commands in different operating systems (Windows, Linux, and UNIX) for managing files, directories, permissions, and user interactions through a terminal or command-line interface.

- **Objectives:**

1. To learn and practice fundamental command-line operations for file and directory management.
2. To explore and utilize user and permission management commands effectively.
3. To enhance system administration skills by working with commands across different operating systems.

- **Requirements:**

Hardware Requirements:

- **Processor**: Multi-core CPU, Intel Core i3 (3.0 GHz) or higher
- **RAM**: Minimum 4 GB (8 GB recommended for optimal performance)
- **Storage**: 100 GB HDD or SSD (Solid State Drive) for faster access
- **Network Interface**: Ethernet or Wi-Fi adapter for connectivity

Software Requirements:

- **Operating System**: Windows 10/11, Linux (Ubuntu 20.04/CentOS 8), UNIX-based OS
- **Command-line Interface**: PowerShell or Command Prompt (Windows), Terminal (Linux/UNIX)
- **Text Editor**: Nano, Vim, or Visual Studio Code for file editing
- **Administrative Privileges**: Superuser (Linux/UNIX) or Administrator (Windows) access

- **Theory:**

In system administration, command-line interfaces (CLI) are essential tools for managing and interacting with operating systems like Windows, Linux, and UNIX. Commands allow users to perform various tasks such as navigating directories, managing files, controlling permissions, and monitoring system performance. Each operating system provides a set of built-in commands, such as ‘man’, ‘ls’, ‘cd’, ‘mkdir’, and ‘chmod’, to facilitate efficient system management. Understanding these commands and their syntax is crucial for automating tasks, enhancing security, and ensuring optimal system functionality. This practical aims to develop foundational skills in executing and applying basic commands across different platforms.

❖ **Commands:**

1. **Display User Manual of a Command**
   - Functionality: Shows the manual page with details about a command's usage, options, and arguments.
   - Syntax: man <command>
   - Example: man ls

2. **Change Current Working Directory**.
   - Functionality: Changes the terminal's current working directory.
   - Syntax: cd <directory-path>
   - Example: cd /home/user/Documents.

3. **List Contents of the Current Directory.**
   - Functionality: Lists all files and directories in the current location.
   - Syntax: ls
   - Example: ls

4. **Read/Modify/Concatenate Text Files.**
   - Functionality: Displays or manipulates file content.
   - Syntax:
     - Read: cat <filename>
     - Modify: 'nano <filename>
     - Concatenate: cat <file1> <file2> > <outputfile>

5. **Create a New Directory**.
   - Functionality: Creates a new directory at the specified path.
   - Syntax: mkdir <directory-name
   - Example: mkdir newdir

6. **Display Current Working Directory.**
   - Functionality: Prints the current directory path.
   - Syntax: pwd
   - Example: pwd

7. **Write Arguments to Standard Output.**
   - Functionality: Prints the provided string or variables.
   - Syntax: echo <arguments>
   - Example: echo Hello World

8. **Remove a File**.
   - Functionality: Deletes a specified file.
   - Syntax: rm <filename>
   - Example: rm file.txt

9. **Delete a Directory.**
   - Functionality: Removes an empty directory.
   - Syntax: rmdir <directory-name>
   - Example: rmdir olddir

10. **Copy a File or Directory.**
    - Functionality: Copies a file or directory to a destination.
    - Syntax: cp <source> <destination>
    - Example: cp file.txt backup/
11. **Switch to Root User.**
    - Functionality: Gains root privileges temporarily.
    - Syntax: sudo su
    - Example: sudo s

12. **Move Files or Directories.**
    - Functionality: Moves or renames files and directories.
    -Syntax: mv <source> <destination>
    - Example: mv file.txt newdir/
13. **Search for a String in a File.**
    - Functionality: Searches for a specific word or pattern in a file.
    - Syntax: grep "<string>" <file>
    - Example: grep "error" log.txt

14. **Print Top N Lines of a File.**
    - Functionality: Displays the first N lines of a file.
    - Syntax: head -n <N> <file>
    - Example: ‘head -n 10 file.txt’

15. **Print Last N Lines of a File.**
    - Functionality: Displays the last N lines of a file.
    - Syntax: tail -n <N> <file>
    - Example: ‘tail -n 10 file.txt

16. **Remove Read Permission from Owner.**
   - Functionality: Revokes the owner's read permission for a file.
   - Syntax: chmod u-r <filename>
   - Example: chmod u-r file.txt

17. **Change Specific Permissions.**
   - Functionality: Sets or removes specific file permissions.
   -Syntax: chmod u+r,w-x,g+w <filename>
   - Example: chmod u+r,w-x,g+w file.txt

18. **Add Write Permission to Owner, None to Others**.
   -Functionality: Allows write access for the owner only.
   - Syntax: chmod u+w,o-rwx <filename>
   - Example: chmod u+w,o-rwx file.txt

19. **Assign Permissions to Users.**
   - Functionality: Modifies file access for users, groups, and others.
   - Syntax: chmod u+rwx,g+rx,o+r <filename>
   - Example: 'chmod u+rwx,g+rx,o+r file.txt

20. **Assign R/W/X to Others.**
   - Functionality: Gives read, write, and execute permissions to others.
   - Syntax: chmod o+rwx <filename>
   - Example: chmod o+rwx file.txt

21. **Remove All Permissions from All Users.**
   - Functionality: Clears all permissions on a file.
   - Syntax: 'chmod a-rwx <filename>
   - Example: 'chmod a-rwx file.txt

22. **Remove Read Permission Using Absolute Mode.**
   - Functionality: Uses numeric mode to restrict read access.
   - Syntax: chmod 700 <filename>
   - Example: chmod 700 file.txt

23. **Set R/W for Owner, None for Group/Other.**
   - Functionality: Assigns permissions in numeric mode.
   - Syntax: chmod 600 <filename>
   - Example: chmod 600 file.txt'

24. **Add Execute for Owner, Read for Group/Others.**
   - Functionality: Adds execution and read access.
   - Syntax: chmod u+x,g+r,o+r <filename>

- Example: chmod u+x,g+r,o+r file.txt

25. **Add Execute Permission to All Users.**
    - Functionality: Enables execution by everyone.
    - Syntax: chmod a+x <filename>
    - Example: chmod a+x script.sh

❖ **Conclusion:** In conclusion, understanding and using essential operating system commands like **'ls', 'cd', 'cp', 'mv', and 'chmod'** enables efficient file management, navigation, and permission control. Tools like **'grep', 'head', and 'tail'** enhance data processing. Mastery of these commands improves system administration, task automation, and overall system security and performance.

❖ **Discussion Questions:**

1. **What is the significance of the pwd command in a Linux environment?**
   Answer: The pwd (print working directory) command displays the absolute path of the current working directory. It helps verify the user's present location in the file system. Syntax: pwd.

2. **Explain the function of the cp command and its common options.**
   Answer: The cp command copies files or directories. Syntax: cp <source> <destination>. Options like -r copy directories recursively, and -i prompts before overwriting.

3. **How does chmod 700 affect file permissions, and what does each digit represent?**
   Answer: chmod 700 grants full permissions (read, write, execute) to the owner and no permissions to others. The digits represent permissions for the owner, group, and others, respectively.

4. **Describe the difference between head and tail commands in Linux. Answer:** The head command displays the first N lines of a file, while tail shows the last N lines. Both accept the -n option to specify the number of lines.

5. **What is the purpose of the grep command, and how is it used with regular expressions?**
   Answer: The grep command searches for patterns within files using regular expressions. Syntax: grep <pattern> <file>. It supports options like -i for case-insensitive search and -v to invert the match.

❖ **References**:

https://ubuntu.com/tutorials/command-line-for-beginners#1-overview
https://www.geeksforgeeks.org/25-basic-ubuntu-commands/

Date:_____/_____/2026

Signature
Course Coordinator
B.Tech CSE(DS) Sem: 4
/ 2025-26

S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 03

Aim: Automate student marksheet generation, system information display, Fibonacci and prime number generation, and file management operations using shell scripts to enhance computational efficiency and user interaction.

Name:

USN:

Semester / Year: Academic

Session: Date of

Performance: Date of

Submission:

❖ **Aim:** Automate student marksheet generation, system information display, Fibonacci and prime number generation, and file management operations using shell scripts to enhance computational efficiency and user interaction.

❖ **Tasks to be done in this Practical.**

a) Write a shell script to generate mark- sheet of a student. Take 3 subjects, calculate and display total marks, percentage and Class obtained by the student.

b) Write a menu driven shell script which will print the following menu and execute the given task.
- ➢ Display calendar of current month.
- ➢ Display today's date and time.
- ➢ Display usernames those are currently logged in the system.
- ➢ Display your terminal number

c) Write a shell script which will generate first n Fibonacci numbers like: 1, 1, 2, 3, 5, 13

d) Write a shell script which will accept a number b and display first n prime numbers as output.

e) Write menu driven program for file handling activity
- ➢ Creation of file.
- ➢ Write content in the file.
- ➢ Upend file content.
- ➢ Delete file content

❖ **Objectives:**

1. Automate marksheet generation with total, percentage, and class classification.
2. Develop menu-driven scripts for system information and file operations.
3. Generate Fibonacci and prime numbers for user-defined inputs.

❖ **Requirements:**

✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

✓ **Software Requirements:**

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

❖ **Theory:**

Shell scripting is a powerful way to automate repetitive tasks and manage system operations efficiently. It allows users to write programs using shell commands and scripting constructs. Shell scripts are interpreted line-by-line by a shell interpreter, making them ideal for administrative tasks, file management, and system automation. This practical encompasses a variety of real-world scenarios that demonstrate the utility of shell scripting for computing tasks and resource management.

1. Marksheet Generation

This script takes input marks for three subjects, calculates the total marks, percentage, and determines the class of the student based on predefined conditions. Conditional statements (if-else) are used to classify the performance into distinction, first class, second class, or fail. This exercise emphasizes the use of arithmetic operations and decision-making constructs.

Key concepts include:

- Reading user input using read
- Arithmetic operations with $((expression))
- Conditional statements for decision-making

2. Menu-Driven Script for System Information

Menu-driven scripts enhance user interaction by presenting a list of options for performing different tasks. In this practical, options are provided to display the calendar of the current month, the current date and time, logged-in users, and the terminal number. The script utilizes looping constructs (while) and case statements for structured flow control.

Commands used:

- cal for displaying the calendar
- date for showing current date and time
- who to list logged-in users
- tty to identify the terminal

3. Fibonacci Number Generation

Fibonacci numbers are a sequence where each term is the sum of the two preceding ones. The script uses iterative constructs (for loop) to generate n terms based on user input. This practical illustrates the use of loop control and variable swapping to generate series data efficiently.

4. Prime Number Display

This script accepts an integer n and outputs the first n prime numbers. A nested loop checks divisibility to determine if a number is prime. The practical demonstrates logic building for number-theoretic operations using loops and conditionals.

5. Menu-Driven File Management

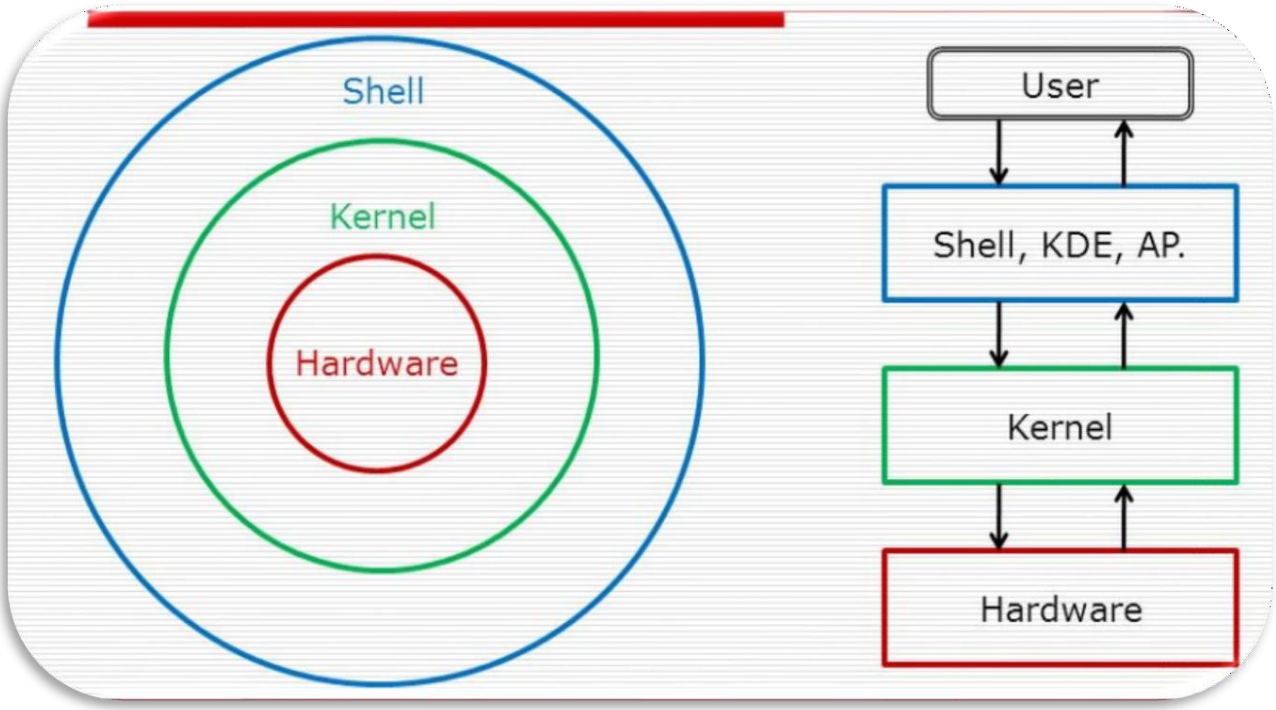
The file handling script enables users to create, write, append, and delete file content. The case construct manages different file operations.

Commands include:

- touch to create files
- cat for writing and appending content
- rm for deleting files

This exercise emphasizes text manipulation, input handling, and file control mechanisms in Unix-like environments.

Diagrammatical View of Shell



❖ **CODES**

1. **Write a shell script to generate mark- sheet of a student. Take 3 subjects, calculate and display total marks, percentage and Class obtained by the student.**

Output 1:

2. **Write a menu driven shell script which will print the following menu and execute the given task.**
   - **Display calendar of current month.**
   - **Display today's date and time.**
   - **Display usernames those are currently logged in the system.**
   - **Display your terminal number**

Output 2:

3. **Write a shell script which will generate first n Fibonacci numbers like: 1, 1, 2, 3, 5, 13**

4. **Write a shell script which will accept a number b and display first n prime numbers as output.**

Output 4:

5. Write menu driven program for file handling activity
   - Creation of file.
   - Write content in the file.
   - Upend file content.
   - Delete file content

Output 5:

❖ **Conclusion**: In this practical, we conclude that shell scripting efficiently automates tasks like marksheet generation, system information display, number computations, and file management, enhancing system operations and user interaction through command-line utilities.

❖ **Discussion Questions:**

1. **What is the purpose of using shell scripting in this practical?**
   Ans: Shell scripting in this practical automates tasks like marksheet generation, Fibonacci sequence calculation, system information display, and file management. It improves efficiency by reducing manual interventions and allows easy automation of routine tasks using simple commands and logic.
2. **Which command is used to display the current date and time?**
   Ans: The date command is used to display the current date and time. It provides a formatted output, showing the system's current time, date, and related information depending on the user's locale settings.
3. **How does the script calculate the Fibonacci sequence?**
   Ans: The Fibonacci sequence script uses a loop to iteratively calculate each number by summing the previous two numbers in the sequence, starting from 0 and 1. This continues until the specified number of terms is generated, making use of simple arithmetic operations.
4. **Which command is used to create a file in the file management script? Ans:** The touch command is used to create an empty file or update the timestamp of an existing file. This is a simple and effective way to initiate a file before performing further operations like writing or appending content.
5. **How does the prime number script determine if a number is prime? Ans:** The prime number script checks for divisibility by iterating through potential divisors up to the square root of the number. If no divisor other than 1 and the number itself is found, the number is classified as prime.

❖ **References:** *https://www.tutorialspoint.com/unix/shell_scripting.html*
https://www.javatpoint.com/shell-scripting-tutorial

Date:_____/_____/2026

Signature
Course Coordinator
B.Tech CSE(DS)

S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 04

Aim: Implement process creation using the fork() system call in Linux to generate parent and child processes, retrieve their PID and PPID, and analyse system behavior by creating orphan and zombie processes.

Name:

USN:

Semester / Year:

Academic Session:

Date of Performance:

Date of Submission:

- **Aim:** Implement process creation using the fork() system call in Linux to generate parent and child processes, retrieve their PID and PPID, and analyse system behaviour by creating orphan and zombie processes.

- **Tasks to be done in this Practical.**

1. Adam is working in an IT company. He has been given a task to reduce the load of a system by killing some of the processes running in the LINUX operating system. Which commands will he use to complete the given task with the help of the following operation?
   - Kill processes by name
   - Kill a process based on the process name
   - Kill a single process at a time with the given process ID

2. Write a program for process creation using C
   - Orphan Process
   - Zombie Process

3. Create the process using fork () system call.
   - Child Process creation
   - Parent process creation
   - PPID and PID

- **Objectives:**

a. Implement process creation using the fork() system call to generate parent and child processes.
b. Retrieve and display the Process ID (PID) and Parent Process ID (PPID) for both processes.
c. Analyse system behaviour by creating and observing orphan and zombie processes.

- **Requirements:**

✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz.
- RAM: 512 MB or higher.
- Storage: 100 MB free space.

✓ **Software Requirements:**

- Operating System: Linux/Unix-based.
- Compiler: GCC Compiler.
- Text Editor: Nano, Vim, or any preferred editor.

- **Theory:**

In many operating systems, the fork system call is an essential operation. The fork system call allows the creation of a new process. When a process calls the fork(), it duplicates itself, resulting in two processes running at the same time. The new process that is created is called a child process. It is a copy of the parent process. The fork system call is required for process creation and enables many important features such as parallel processing, multitasking, and the creation of complex process hierarchies.
It develops an entirely new process with a distinct execution setting. The new process has its own address space, and memory, and is a perfect duplicate of the caller process.

Basic Terminologies Used in Fork System Call in Operating System

- **Process:** In an operating system, a process is an instance of a program that is currently running. It is a separate entity with its own memory, resources, CPU, I/O hardware, and files.
- **Parent Process:** The process that uses the fork system call to start a new child process is referred to as the parent process. It acts as the parent process's beginning point and can go on running after the fork.
- **Child Process:** The newly generated process as a consequence of the fork system call is referred to as the child process. It has its own distinct process ID (PID), and memory, and is a duplicate of the parent process.
- **Process ID:** A process ID (PID) is a special identification that the operating system assigns to each process.
- **Copy-on-Write:** The fork system call makes use of the memory management strategy known as copy-on-write. Until one of them makes changes to the shared memory, it enables the parent and child processes to share the same physical memory. To preserve data integrity, a second copy is then made.
- **Return Value:** The fork system call's return value gives both the parent and child process information. It assists in handling mistakes during process formation and determining the execution route.

Process Creation

When the fork system call is used, the operating system completely copies the parent process to produce a new child process. The memory, open file descriptors, and other pertinent properties of the parent process are passed down to the child process. The child process, however, has a unique execution route and PID.
The copy-on-write method is used by the fork system call to maximize memory use. At first, the physical memory pages used by the parent and child processes are the same. To avoid unintentional changes, a separate copy is made whenever either process alters a shared memory page.The return value of the fork call can be used by the parent to determine the execution path of the child process. If it returns 0 then it is executing the child process, if it returns -1 then there is some error; and if it returns some positive value, then it is the PID of the child process.

Advantages of Fork System Call

- **Creating new processes** with the fork system call facilitates the running of several tasks concurrently within an operating system. The system's efficiency and multitasking skills are improved by this concurrency.
- **Code reuse:** The child process inherits an exact duplicate of the parent process, including every code segment, when the fork system call is used. By using existing code, this feature encourages code reuse and streamlines the creation of complicated programmes.
- **Memory Optimisation:** When using the fork system call, the copy-on-write method optimises the use of memory. Initial memory overhead is minimised since parent and child processes share the same physical memory pages. Only when a process changes a shared memory page, improving memory efficiency, does copying take place.
- Process isolation is achieved by giving each process started by the fork system call its own memory area and set of resources. System stability and security are improved because of this isolation, which prevents processes from interfering with one another.

Disadvantages of Fork System Call

- **Memory Overhead:** The fork system call has memory overhead even with the copy-on-write optimisation. The parent process is first copied in its entirety, including all of its memory, which increases memory use.
- **Duplication of Resources:** When a process forks, the child process duplicates all open file descriptors, network connections, and other resources. This duplication may waste resources and perhaps result in inefficiencies.
- **Communication complexity:** The fork system call generates independent processes that can need to coordinate and communicate with one another. To enable data transmission across processes, interprocess communication methods, such as pipes or shared memory, must be built, which might add complexity.
- **Impact on System Performance:** Forking a process duplicates memory allocation, resource management, and other system tasks. Performance of the system may be affected by this, particularly in situations where processes are often started and stopped.

❖ **CODES**

- Adam is working in an IT company. He has been given a task to reduce the load of a system by killing some of the processes running in the LINUX operating system. Which commands will he use to complete the given task with the help of the following operation?

  - Kill processes by name

```
samiy@DESKTOP-HH652PV UCRT64 ~
$ sleep 500 &
[1] 1967

samiy@DESKTOP-HH652PV UCRT64 ~
$ ps
      PID    PPID    PGID     WINPID   TTY         UID    STIME COMMAND
     1967    1920    1967       1576  pty1      197609 17:57:35 /usr/bin/slee
     1945       1    1945       4488  ?         197609 17:56:52 /usr/bin/mint
     1946    1945    1946       8916  pty0      197609 17:56:52 /usr/bin/bash
     1920    1919    1920      11912  pty1      197609 17:51:34 /usr/bin/bash
     1968    1920    1968       9036  pty1      197609 17:59:13 /usr/bin/ps
     1919       1    1919      15124  ?         197609 17:51:34 /usr/bin/mint

samiy@DESKTOP-HH652PV UCRT64 ~
$ pkill sleep
[1]+  Terminated                 sleep 500

samiy@DESKTOP-HH652PV UCRT64 ~
$ pgrep sleep

samiy@DESKTOP-HH652PV UCRT64 ~
$
```

  - Kill a process based on the process name

```
samiy@DESKTOP-HH652PV UCRT64 ~
$ sleep 500 &
[1] 1972

samiy@DESKTOP-HH652PV UCRT64 ~
$ pgrep -a sleep
1972 sleep 500

samiy@DESKTOP-HH652PV UCRT64 ~
$ pkill sleep

samiy@DESKTOP-HH652PV UCRT64 ~
$
```

- Kill a single process at a time with the given process ID

```
samiy@DESKTOP-HH652PV UCRT64 ~
$ sleep 500 &
[1] 1977

samiy@DESKTOP-HH652PV UCRT64 ~
$ ps
      PID    PPID    PGID     WINPID   TTY         UID    STIME COMMAND
     1945       1    1945       4488  ?         197609 17:56:52 /usr/bin/mintty
     1946    1945    1946       8916  pty0      197609 17:56:52 /usr/bin/bash
     1920    1919    1920      11912  pty1      197609 17:51:34 /usr/bin/bash
     1977    1920    1977       6340  pty1      197609 19:07:58 /usr/bin/sleep
     1978    1920    1978       7220  pty1      197609 19:08:07 /usr/bin/ps
     1919       1    1919      15124  ?         197609 17:51:34 /usr/bin/mintty

samiy@DESKTOP-HH652PV UCRT64 ~
$ pgrep -a sleep
1977 sleep 500

samiy@DESKTOP-HH652PV UCRT64 ~
$ kill 1977

samiy@DESKTOP-HH652PV UCRT64 ~
$
```

- **Orphan.c**

```
GNU nano 8.7                    orphan.c                    Modified
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid = fork();

    if (pid > 0)
    {
        printf("Parent process terminating...\n");
    }
    else
    {
        sleep(5);
        printf("Child process running after parent termination\n");
        printf("Child PID: %d\n", getpid());
        printf("New Parent PID: %d\n", getppid());
    }

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

```
samiy@DESKTOP-HH652PV MSYS ~
$ pacman -S gcc
resolving dependencies...
looking for conflicting packages...

Packages (8) binutils-2.45.1-1  isl-0.27-1  mpc-1.3.1-1  msys2-runtime-devel-3.6.6-1
             msys2-w32api-headers-13.0.0.r0.gde62c283f-1  msys2-w32api-runtime-13.0.0.r0.gde62c283f-1
             windows-default-manifest-6.4-2  gcc-15.2.0-1

Total Installed Size:  442.31 MiB

:: Proceed with installation? [Y/n] Y
:: Retrieving packages...
 isl-0.27-1-x86_64 is up to date
(8/8) checking keys in keyring                       [#################################] 100%
(8/8) checking package integrity                     [#################################] 100%
(8/8) loading package files                          [#################################] 100%
(8/8) checking for file conflicts                    [#################################] 100%
(8/8) checking available disk space                  [#################################] 100%
:: Processing package changes...
(1/8) installing binutils                            [#################################] 100%
(2/8) installing isl                                 [#################################] 100%
(3/8) installing mpc                                 [#################################] 100%
(4/8) installing msys2-runtime-devel                 [#################################] 100%
(5/8) installing msys2-w32api-headers                [#################################] 100%
(6/8) installing msys2-w32api-runtime                [#################################] 100%
(7/8) installing windows-default-manifest            [#################################] 100%
(8/8) installing gcc                                 [#################################] 100%
:: Running post-transaction hooks...
(1/1) Updating the info directory file...

samiy@DESKTOP-HH652PV MSYS ~
$ nano orphan.c

samiy@DESKTOP-HH652PV MSYS ~
$ gcc orphan.c -o orphan

samiy@DESKTOP-HH652PV MSYS ~
$ ./orphan
Parent process terminating...

samiy@DESKTOP-HH652PV MSYS ~
$ Child process running after parent termination
Child PID: 2187
New Parent PID: 1
$
```

- **Zombie.c**

```
  GNU nano 8.7                                zombie.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid = fork();

    if (pid > 0)
    {
        printf("Parent process terminating...\n");
    }
    else if (pid == 0)
    {
        sleep(5);
        printf("Child process running after parent termination\n");
        printf("Child PID: %d\n", getpid());
        printf("New Parent PID: %d\n", getppid());
    }
    else
    {
        printf("Fork failed\n");
    }

    return 0;
}

^G Help      ^O Write Out   ^F Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo     M-6 Copy
```

```
samiy@DESKTOP-HH652PV MSYS ~
$ nano zombie.c

samiy@DESKTOP-HH652PV MSYS ~
$ gcc zombie.c -o zombie

samiy@DESKTOP-HH652PV MSYS ~
$ ./zombie
Parent process terminating...

samiy@DESKTOP-HH652PV MSYS ~
$ Child process running after parent termination
Child PID: 2198
New Parent PID: 1
$
```

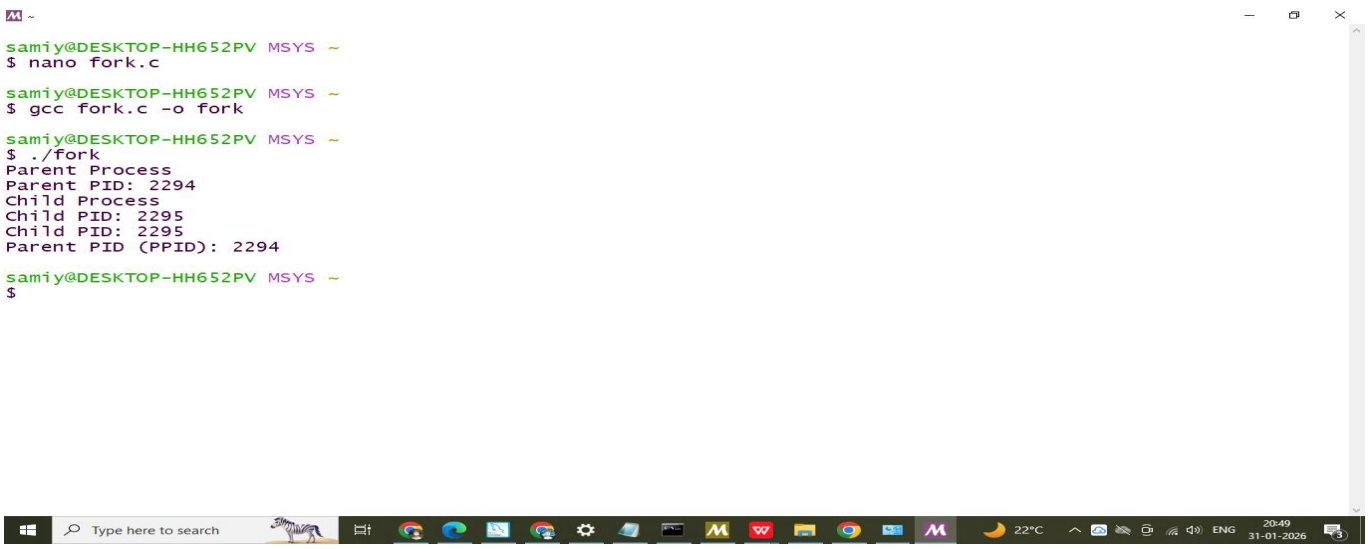
- Create the process using fork () system call.
  - Child Process creation
  - Parent process creation
  - PPID and PID

```
  GNU nano 8.7                                   fork.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
    }
    else if (pid == 0)
    {
        // Child process
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID (PPID): %d\n", getppid());
    }
    else
    {
        // Parent process
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
    }

                                   [ Read 29 lines ]
^G Help      ^O Write Out   ^F Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo
```



```
samiy@DESKTOP-HH652PV MSYS ~
$ nano fork.c

samiy@DESKTOP-HH652PV MSYS ~
$ gcc fork.c -o fork

samiy@DESKTOP-HH652PV MSYS ~
$ ./fork
Parent Process
Parent PID: 2294
Child Process
Child PID: 2295
Child PID: 2295
Parent PID (PPID): 2294

samiy@DESKTOP-HH652PV MSYS ~
$
```

- **Conclusion**: In this practical, we conclude that Linux commands like pkill, killall, and kill effectively reduce system load by terminating processes based on their name or ID, improving system performance and stability.

- **Discussion Questions:**

1. What is the concept of pointers in C?

Ans: A pointer in C is a variable that stores the memory address of another variable. It enables direct access to memory, facilitating dynamic memory management and efficient function calls, especially for large data structures.

2. What is the difference between malloc and calloc in C?

Ans: malloc allocates uninitialized memory, while calloc allocates memory and initializes all bits to zero. Use malloc when initialization is not required and calloc when zeroed memory is needed.

3. What is the purpose of the fork() system call in Unix?

Ans: fork() creates a new process by duplicating the parent process. It allows for parallel execution of processes, where the child process gets a copy of the parent's address space.

4. What is the difference between struct and union in C?

Ans: A struct allocates separate memory for each member, while a union shares the same memory space for all members, saving memory but allowing only one member to hold a value at a time.

5. What does grep do in Linux?

Ans: grep searches for specific patterns in files and outputs the lines containing those patterns. It's a powerful tool for text processing and log analysis in Linux environments.

- **References:**

https://www.geeksforgeeks.org/fork-system-call-in-operating- system/?ref=ml_lbp
https://www.scaler.com/topics/fork-system-call/

Date:_____/_____/2026

Signature Course Coordinator B.Tech CSE(DS)
Sem: 4 / 2025-26

S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 05

Aim: Write a program to implement Shortest Job First (SJF) Preemptive Scheduling for three processes and calculate the total context switches and average waiting time. The processes have burst times 10ns, 20ns, and 30ns, arriving at 0ns, 2ns, and 6ns, respectively.

Name:

USN:

Semester / Year: Academic

Session: Date of

Performance: Date of

Submission:

- **Aim:** Write a program to implement Shortest Job First (SJF) Preemptive Scheduling for three processes and calculate the total context switches and average waiting time. The processes have burst times 10ns, 20ns, and 30ns, arriving at 0ns, 2ns, and 6ns, respectively.

- **Objectives:**
  Understand SJF Preemptive Scheduling: Implement the **Shortest Job First (SJF) Preemptive Scheduling** algorithm to manage CPU execution efficiently.
  Calculate Context Switches: Determine the total number of context switches required for the given set of processes.
  Evaluate Waiting Time: Compute the **average waiting time** for all processes before getting CPU execution.

- **Requirements:**
  - ✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

  - ✓ **Software Requirements:**

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

- **Theory:**

CPU Scheduling in Operating Systems

Introduction

Scheduling is the method by which processes are given access to the CPU. Efficient scheduling is essential for optimal system performance and user experience. There are two primary types of CPU scheduling: **Preemptive Scheduling** and **Non-Preemptive Scheduling**.
Understanding the differences between these scheduling types helps in designing and choosing the right scheduling algorithms for different operating systems.

---

1. **Preemptive Scheduling**

In **Preemptive Scheduling**, the operating system can interrupt or preempt a running process to allocate CPU time to another process, typically based on priority or time-sharing policies. A process can be switched from the **running state to the ready state** at any time.

Algorithms Based on Preemptive Scheduling:

- **Round Robin (RR)**
- **Shortest Remaining Time First (SRTF)**
- **Priority Scheduling (Preemptive version) Example:**

In the following case, **P2 is preempted at time 1** due to the arrival of a higher- priority process.

Advantages of Preemptive Scheduling:

✔ Prevents a process from monopolizing the CPU, improving system reliability.

✔ Enhances **average response time**, making it beneficial for multi- programming environments.

✔ Used in modern operating systems like **Windows, Linux, and macOS**. Disadvantages of Preemptive Scheduling:

✗ More complex to implement.

✗ Involves **overhead** for suspending a running process and switching contexts.

✗ **May cause starvation** if low-priority processes are frequently preempted.

✗ Can create **concurrency issues**, especially when accessing shared resources.

---

2. **Non-Preemptive Scheduling**

In **Non-Preemptive Scheduling**, a running process cannot be interrupted by the operating system. It continues executing until it **terminates** or **enters a waiting state** voluntarily.

Algorithms Based on Non-Preemptive Scheduling:

- **First Come First Serve (FCFS)**
- **Shortest Job First (SJF - Non-Preemptive)**
- **Priority Scheduling (Non-Preemptive version) Example:**

Below is a **Gantt Chart** based on the **FCFS algorithm**, where each process executes fully before the next one starts.

Advantages of Non-Preemptive Scheduling:

✔ **Easy to implement** in an operating system (used in older versions like Windows 3.11 and early macOS).

✔ **Minimal scheduling overhead** due to fewer context switches.

✔ **Less computational resource usage**, making it more efficient for simpler systems.

Disadvantages of Non-Preemptive Scheduling:

✗ **Risk of Denial of Service (DoS) attacks**, as a process can monopolize the CPU.

✗ **Poor response time**, especially in multi-user systems.

---

3. **Differences Between Preemptive and Non-Preemptive Scheduling**

Parameter	Preemptive Scheduling	Non-Preemptive Scheduling
Basic Concept	CPU time is allocated for a **limited time**.	CPU is held until process **terminates** or enters waiting state.
Interrupts	Process **can be interrupted**.	Process **cannot be interrupted**.
Starvation	Frequent high-priority processes may starve low- priority ones.	A long-running process can starve later-arriving shorter processes.
Overhead	Higher overhead due to frequent **context switching**.	Minimal overhead.
Flexibility	More flexible (critical processes get priority).	Rigid scheduling approach.
Response Time	Faster response time.	Slower response time.
Process Control	**OS has more control** over scheduling.	OS has **less control** over scheduling.
Concurrency Issues	**Higher**, as processes may be preempted during shared resource access.	**Lower**, as processes run to completion.
Examples	Round Robin, SRTF.	FCFS, Non-Preemptive SJF.

---

4. **Frequently Asked Questions (FAQs)**

a. How is priority determined in Preemptive scheduling?

Ans: Preemptive scheduling systems assign priority based on **task importance, deadlines, or urgency**. Higher-priority tasks execute before lower-priority ones.

b. What happens in non-preemptive scheduling if a process does not yield the CPU?

Ans: If a process does not voluntarily yield the CPU, it can lead to **starvation or deadlock**, where other tasks are unable to execute.

c. Which scheduling method is better for real-time systems?
Ans: Preemptive scheduling is better for **real-time systems**, as it allows high- priority tasks to execute immediately.

❖ **CODE**

❖ **Output:**

Conclusion: Preemptive scheduling offers better responsiveness but adds complexity, while non-preemptive scheduling is simpler but may cause inefficiencies. The choice depends on system needs, with preemptive suited for multitasking and non-preemptive for low-overhead scenarios.

❖ **Discussion Questions:**

1. **What is the key difference between preemptive and non-preemptive scheduling?**
   Ans: Preemptive scheduling allows the CPU to be taken away from a process before it finishes execution, whereas non-preemptive scheduling ensures a process runs until completion or voluntary release of the CPU.
2. **Why does preemptive scheduling require context switching?**
   Ans: Preemptive scheduling interrupts running processes, requiring the

system to save the current state (context) and load the next process, leading to context switching overhead.

3. **Which CPU scheduling algorithm is most suitable for real-time systems and why?**
   Ans: Preemptive priority scheduling is best for real-time systems as it ensures high-priority tasks get immediate CPU access, reducing response time for critical tasks.
4. **What is starvation in CPU scheduling, and how can it be prevented? Ans:** Starvation occurs when low-priority processes wait indefinitely due to frequent arrival of high-priority tasks. It can be prevented using **aging**, which gradually increases a process's priority over time.
5. **Why is the Round Robin scheduling algorithm preferred in time- sharing systems?**
   Ans: Round Robin ensures **fair CPU allocation** by assigning time slices (quantum) to each process, preventing monopolization and providing a balanced response time in multi-user environments.

❖ **References:**

https://www.geeksforgeeks.org/preemptive-and-non-preemptive-scheduling/

Date:_____/_____/2026

Signature Course Coordinator B.Tech CSE(DS)
Sem: 4 / 2025-26

S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 06

Aim: Considered there are N philosophers seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. One chopstick may be picked up by any one of its adjacent followers but not both.
Write a program to solve the problem using process synchronization technique.

Name:

USN:

Semester / Year: Academic

Session: Date of

Performance: Date of

Submission:

Aim: Considered there are N philosophers seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. One chopstick may be picked up by any one of its adjacent followers but not both.

Write a program to solve the problem using process synchronization technique.

❖ **Objectives:**

a) T**o implement process synchronization** using semaphores or mutex locks to prevent deadlock and ensure philosophers can eat without conflicts.
b) **To apply the Dining Philosophers Problem** as a classic example of synchronization in concurrent programming.
c) **To ensure no two adjacent philosophers** pick up the same chopstick simultaneously, avoiding resource contention.
d) **To demonstrate deadlock prevention** and starvation avoidance using techniques like ordering of resource allocation or introducing an arbitrator.

❖ **Requirements:**

✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

✓ **Software Requirements:**

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

❖ **Theory:**

The Dining Philosopher Problem states that K philosophers are seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. One chopstick may be picked up by any one of its adjacent followers but not both.

Semaphore Solution to Dining Philosopher

Each philosopher is represented by the following pseudocode:

```
process P[i] while
 true do
   {  THINK;
      PICKUP(CHOPSTICK[i], CHOPSTICK[i+1 mod 5]); EAT;
      PUTDOWN(CHOPSTICK[i], CHOPSTICK[i+1 mod 5])
   }
```

There are three states of the philosopher: **THINKING, HUNGRY, and EATING**. Here there are two semaphores: Mutex and a semaphore array for the philosophers. Mutex is used such that no two philosophers may access the pickup or put it down at the same time. The array is used to control the behavior of each philosopher. But, semaphores can result in deadlock due to programming errors.

Outline of a philosopher process:

```
Var successful: boolean; repeat
      successful:= false; while (not
      successful)

      if both forks are available then lift the
      forks one at a time; successful:= true;

     if successful = false then
     block(Pi);
     {eat}

     put down both forks;

     if left neighbor is waiting for his right fork then
     activate (left neighbor);
```

```
    if right neighbor is waiting for his left fork then
    activate( right neighbor);
  {think} forever
```

The steps for the Dining Philosopher Problem solution using semaphores are as follows:

1. Initialize the semaphores for each fork to 1 (indicating that they are available).
2. Initialize a binary semaphore (mutex) to 1 to ensure that only one philosopher can attempt to pick up a fork at a time.
3. For each philosopher process, create a separate thread that executes the following code:

- **While true:**
  - Think for a random amount of time.
  - Acquire the mutex semaphore to ensure that only one philosopher can attempt to pick up a fork at a time.
  - Attempt to acquire the semaphore for the fork to the left.
- If successful, attempt to acquire the semaphore for the fork to the right.
- If both forks are acquired successfully, eat for a random amount of time and then release both semaphores.
- If not successful in acquiring both forks, release the semaphore for the fork to the left (if acquired) and then release the mutex semaphore and go back to thinking.

4. Run the philosopher threads concurrently.

By using semaphores to control access to the forks, the Dining Philosopher Problem can be solved in a way that avoids deadlock and starvation. The use of the mutex semaphore ensures that only one philosopher can attempt to pick up a fork at a time, while the use of the fork semaphores ensures that a philosopher can only eat if both forks are available.

Overall, the Dining Philosopher Problem solution using semaphores is a classic example of how synchronization mechanisms can be used to solve complex synchronization problems in concurrent programming.

Implementation of the Dining Philosopher Problem solution using semaphores in Python

CODE:

OUTPUT:

As you can see from the output, each philosopher takes turns thinking and eating, and there is no deadlock or starvation.

Problem with Dining Philosopher

We have demonstrated that no two nearby philosophers can eat at the same time from the aforementioned solution to the dining philosopher problem. The problem with the above solution is that it might result in a deadlock situation. If every philosopher picks their left chopstick simultaneously, a deadlock results, and no philosopher can eat. This situation occurs when this happens.

Some of the solutions include the following:

1. The maximum number of philosophers at the table should not exceed four; in this case, philosopher P3 will have access to chopstick C4; he will then begin eating, and when he is finished, he will put down both chopsticks C3 and C4; as a result, semaphore C3 and C4 will now be incremented to 1.
2. Now that philosopher P2, who was holding chopstick C2, will also have chopstick C3, he will do the same when finished eating, allowing other philosophers to eat.
3. While a philosopher in an odd position should select the right chopstick first, a philosopher in an even position should select the left chopstick and then the right chopstick.
4. A philosopher should only be permitted to choose their chopsticks if both of the available chopsticks

(the left and the right) are available at the same time.

5. All four of the initial philosophers (P0, P1, P2, and P3) should choose the left chopstick before choosing the right, while P4 should choose the right chopstick before choosing the left. As a result, P4 will be forced to hold his right chopstick first because his right chopstick, C0, is already being held by philosopher P0 and its value is set to 0. As a result, P4 will become trapped in an infinite loop and chopstick C4 will remain empty. As a result, philosopher P3 has both the left C3 and the right C4. chopstick available, therefore it will

start eating and will put down both chopsticks once finishes and let others eat which removes the problem of deadlock.

- **Conclusion**: In this practical, we conclude that process synchronization techniques, such as semaphores and mutex locks, effectively manage shared resources in concurrent systems. By implementing the Dining Philosophers Problem, we ensured deadlock prevention, starvation avoidance, and efficient resource allocation, demonstrating the importance of synchronization in multi-process environments.

- **Discussion Questions:**

1. **What is the main problem in the Dining Philosophers Problem?**
   Ans: The main problem is ensuring proper synchronization among philosophers who share limited resources (chopsticks) to avoid deadlock and starvation. Each philosopher needs two adjacent chopsticks to eat, but if all philosophers pick up one chopstick simultaneously and wait for the second, a deadlock can occur. The challenge is to design a solution where philosophers can eat without causing system- wide blocking.

2. **Which synchronization techniques can be used to solve this problem?**
   Ans: Various synchronization techniques can be used, such as mutex locks, semaphores, and monitors. Semaphores can be used to control access to chopsticks, ensuring only one philosopher picks up a chopstick at a time. Monitors provide a structured way to manage concurrency by encapsulating shared resources and conditions for safe execution. Mutex locks ensure that only one philosopher can access a chopstick at a time, preventing race conditions.

3. **How can deadlock be avoided in this problem?**
   Ans: Deadlock can be avoided by implementing strategies such as ensuring that at most (N-1) philosophers pick up chopsticks at a time, enforcing an ordered resource allocation (e.g., odd-numbered philosophers pick left chopstick first while even- numbered philosophers pick right chopstick first), or introducing an arbitrator that grants permission to eat. Another approach is to use a timeout mechanism where a philosopher puts down a chopstick if they cannot acquire the second one within a specific time.

4. **Why is starvation a concern in the Dining Philosophers Problem?**
   Ans: Starvation occurs when a philosopher is perpetually unable to acquire both chopsticks due to other philosophers continuously using them. This can happen if the resource allocation is unfair or if certain philosophers have higher priority access. To prevent starvation, techniques like fair scheduling, priority-based execution, and allowing each philosopher a turn to eat can be used. By ensuring that every philosopher gets a chance to eat in a finite amount of time, starvation can be effectively mitigated.

5. **What is the role of semaphores in solving this problem?**

Ans: Semaphores are used to control access to the shared chopsticks, ensuring proper synchronization between philosophers. A semaphore can be associated with each chopstick, initialized to 1, indicating availability. Before picking up a chopstick, a philosopher performs a "wait" operation, and after finishing eating, they perform a "signal" operation to release the chopsticks. This prevents multiple philosophers from using the same chopstick simultaneously and helps avoid race conditions, ensuring smooth and coordinated execution.

- **References:**
  https://www.geeksforgeeks.org/dining-philosopher-problem-using-semaphores/

Date:_____/_____/2026

Signature Course Coordinator B.Tech CSE(AIML)
Sem: 4 / 2025-26

S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 07

Aim: Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

Name:

USN:

Semester / Year: Academic

Session: Date of

Performance: Date of

Submission:

❖ **Aim:** Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

❖ **Objectives:**

1. Implement the Banker's Algorithm to manage resource allocation and prevent deadlocks.
2. Calculate the current resource need for each process based on allocated and maximum resources.
3. Determine a safe execution sequence for processes if resources are available.

❖ **Requirements:**

✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

✓ **Software Requirements:**

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

❖ **Theory:**

Banker's Algorithm in Operating System

Banker's Algorithm is a resource allocation and deadlock avoidance algorithm used in operating systems. It ensures that a system remains in a safe state by carefully allocating resources to processes while avoiding unsafe states that could lead to deadlocks.

- The Banker's Algorithm is a smart way for computer systems to manage how programs use resources, like memory or CPU time.
- It helps prevent situations where programs get stuck and can not finish their tasks. This condition is known as deadlock.
- By keeping track of what resources each program needs and what's available, the banker algorithm makes sure that programs only get what they need in a safe order.

Why Banker's Algorithm is Named So?

The banker's algorithm is named so because it is used in the banking system to check whether a loan can be sanctioned to a person or not. Suppose there are n number of account holders in a bank and the total sum of their money is S. Let us assume that the bank has a certain amount of money Y. If a person applies for a loan then the bank first subtracts the loan amount from the total money that the bank has (Y) and if the remaining amount is greater than S then only the loan is sanctioned. It is done because if all the account holders come to withdraw their money then the bank can easily do it.

It also helps the OS to successfully share the resources between all the processes. The banker's algorithm uses the notation of a safe allocation state to ensure that granting a resource request cannot lead to a deadlock either immediately or in the future. In other words, the bank would never allocate its money in such a way that it can no longer satisfy the needs of all its customers. The bank would try to be in a safe state always.

Components of the Banker's Algorithm

The following **Data structures** are used to implement the Banker's Algorithm:

Let **'n'** be the number of processes in the system and **'m'** be the number of resource types.

1. **Available**
   - It is a 1-D array of size **'m'** indicating the number of available resources of each type.
   - Available[j] = k means there are **'k'** instances of resource type **Rj**
2. **Max**
   - It is a 2-d array of size '**n*m'** that defines the maximum demand of each process in a system.
   - Max[i, j] = k means process **Pi** may request at most **'k'** instances of resource type **Rj.**
3. **Allocation**
   - It is a 2-d array of size **'n*m'** that defines the number of resources of each type currently allocated to each process.
   - Allocation[i, j] = k means process **Pi** is currently allocated **'k'** instances of resource type **Rj**
4. **Need**
   - It is a 2-d array of size **'n*m'** that indicates the remaining resource need of each process.
   - Need [i, j] = k means process **Pi** currently needs **'k'** instances of resource type **Rj**
   - Need [i, j] = Max [i, j] – Allocation [i, j]

Allocation specifies the resources currently allocated to process Pi and Need specifies the additional resources that process Pi may still request to complete its task.

Banker's algorithm consists of a Safety algorithm and a Resource request algorithm.

Banker's Algorithm

1. ***Active :=*** *Running U Blocked for k = 1 to r:*

   New_request[k] := Requested_resources[requesting_process, k]

2. ***Simulated_allocation :=*** *Allocated_resources for k = 1 to r: // Compute projected allocation state*
   Simulated_allocation[requesting_process, k] :=

   Simulated_allocation[requesting_process, k] + New_request[k]

3. ***Feasible :=*** *true*
   for k = 1 to r: // Check if projected allocation state is feasible if Total_resources[k] < Simulated_total_alloc[k]:

   Feasible := false

4. ***if Feasible =*** *true:*

   while set Active contains a process P1 such that:
   for all k, Total_resources[k] – Simulated_total_alloc[k] >= Max_need[P1, k] – Simulated_allocation[P1, k]:
   Delete P1 from Active for k = 1 to r:

   Simulated_total_alloc[k] :=

   Simulated_total_alloc[k] – Simulated_allocation[P1, k]

5. ***if set Active is empty:***
   for k = 1 to r: // Delete the request from pending requests Requested_resources[requesting_process, k] := 0
   for k = 1 to r: // Grant the request Allocated_resources[requesting_process, k] := Allocated_resources[requesting_process, k] + New_request[k] Total_alloc[k] := Total_alloc[k] + New_request[k]

Safety Algorithm

The algorithm for finding out whether or not a system is in a safe state can be described as follows:

1. Let Work and Finish be vectors of length 'm' and 'n' respectively. Initialize: Work = Available

   Finish[i] = false; for i=1, 2, 3, 4….n

2. Find an i such that both
   - 2.1 Finish[i] = false
   - 2.2 Needi <= Work

   if no such i exists goto step (4)

3. Work = Work + Allocation[i]
   Finish[i] = true goto step (2)

4. if Finish [i] = true for all i then the system is in a safe state.

Resource-Request Algorithm

Let Requesti be the request array for process Pi. Requesti [j] = k means process Pi wants k instances of resource type Rj. When a request for resources is made by process Pi, the following actions are taken:

1. If Requesti <= Needi

Goto step (2) ; otherwise, raise an error condition, since the process has exceeded its maximum claim.

2. If Requesti <= Available

Goto step (3); otherwise, Pi must wait, since the resources are not available.

3. Have the system pretend to have allocated the requested resources to process Pi by modifying the state as follows:

Available = Available – Requesti Allocationi =
Allocationi + Requesti Needi = Needi– Requesti

Example: Considering a system with five processes P0 through P4 and three resources of type A, B, C. Resource type A has 10 instances, B has 5 instances and type C has 7 instances. Suppose at time t0 following snapshot of the system has been taken:

Q.1 What will be the content of the Need matrix?

Need [i, j] = Max [i, j] – Allocation [i, j] So, the content of
Need Matrix is:

Q.2 Is the system in a safe state? If Yes, then what is the safe sequence?

Applying the Safety algorithm on the given system,

Q.3 What will happen if process P1 requests one additional instance of resource type A and two instances of resource type C?

Unsafe State in The Bankers Algorithm

In the context of the Banker's Algorithm, an **unsafe state** refers to a situation in which, all processes in a system currently hold resources within their maximum needs, there is no guarantee that the system can avoid a deadlock in the future. This state is contrasted with a **safe state**, where there exists a sequence of resource allocation and deallocation that guarantees that all processes can complete without leading to a deadlock.

Key Concepts in Banker's Algorithm

- **Safe State:** There exists at least one sequence of processes such that each process can obtain the needed resources, complete its execution, release its resources, and thus allow other processes to eventually complete without entering a deadlock.
- **Unsafe State:** Even though the system can still allocate resources to some processes, there is no guarantee that all processes can finish without potentially causing a deadlock.

Example of Unsafe State

Consider a system with three processes (P1, P2, P3) and 6 instances of a resource. Let's say:

- The available instances of the resource are: 1
- The current allocation of resources to processes is:
  - P1: 2
  - P2: 3
  - P3: 1
- The maximum demand (maximum resources each process may eventually request) is:
  - P1: 4
  - P2: 5
  - P3: 3

In this situation:

Need = Maximum Demand – Allocation

Process	Maximum Demand	Allocation	Need
P1	4	2	2
P2	5	3	2
P3	3	1	2

- P1 may need 2 more resources to complete.
- P2 may need 2 more resource to complete.
- P3 may need 2 more resources to complete.

However, there is only 1 resource available. Even though none of the processes are currently deadlocked, the system is in an unsafe state because there is no sequence of resource allocation that guarantees all processes can complete.

❖ **CODE**

❖ **Output:**

❖ **Conclusion**: In this practical, we conclude that the Banker's Algorithm effectively ensures safe resource allocation by preventing deadlocks. By calculating the current resource need of each process and checking for a safe execution sequence, we can determine whether all processes can complete execution without system instability.

❖ **Discussion Questions:**

1. What is the purpose of the Banker's Algorithm?
Ans: The Banker's Algorithm is used for deadlock avoidance by ensuring safe resource allocation and checking if processes can execute without leading to system instability.

2. How do you calculate the current need of each process? Ans: The current need of a process is calculated as:
Need = MAX - Allocation,
where **MAX** is the maximum required resources, and **Allocation** is the currently allocated resources.

3. What are the conditions for a system to be in a safe state?
Ans: A system is in a safe state if there exists a safe sequence of process execution where each process gets the required resources and completes execution without causing a deadlock.

4. What happens if no safe sequence exists in the Banker's Algorithm? Ans: If no safe sequence exists, the system is in an unsafe state, meaning a deadlock could occur if resources are allocated improperly.

5. What are the key inputs required to implement the Banker's Algorithm? Ans: The key inputs are:
- **Allocated Resources Matrix** (Resources currently allocated to each process)
- **Maximum Resources Matrix** (Maximum resources a process may need)
- **Available Resources Vector** (Resources currently available in the system)

❖ **References:**

https://www.geeksforgeeks.org/bankers-algorithm-in-operating-system-2/ https://chatgpt.com/

Date:____/____/2026

Signature Course Coordinator B.Tech CSE(AIML)
Sem: 4 / 2025-26

S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 08

Aim: A system has RAM which can store four pages simultaneously to satisfy page requests. Write a Program to calculate percentage page hit and page fault if the system uses Least recently used page replacement technique for the page references entered by the user i.e. 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (No of Frames=3).

Name:

USN:

Semester / Year: Academic

Session: Date of

Performance: Date of

Submission:

❖ **Aim:** A system has RAM which can store four pages simultaneously to satisfy page requests. Write a Program to calculate percentage page hit and page fault if the system uses Least recently used page replacement technique for the page references entered by the user i.e. 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (No of Frames=3).

❖ **Objectives:**

1. Implement the Least Recently Used (LRU) page replacement algorithm for memory management.
2. Calculate the number of page hits and page faults for a given page reference sequence.
3. Determine the percentage of page hits and page faults based on the system's frame capacity.

❖ **Requirements:**

✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

✓ **Software Requirements:**

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

❖ **Theory:**

In an operating system, page replacement refers to a scenario in which a page from the main memory should be replaced by a page from the secondary memory. Page replacement occurs due to page faults. The various page replacement algorithms like FIFO, Optimal page replacement, LRU, LIFO, and Random page replacement help the operating system decide which page to replace.

What is Page Replacement in Operating Systems?

Page replacement is needed in the operating systems that use virtual memory using Demand Paging. As we know in Demand paging, only a set of pages of a process is loaded into the memory. This is done so that we can have more processes in the memory at the same time.

When a page that is residing in virtual memory is requested by a process for its execution, the Operating System needs to decide which page will be replaced by this requested page. This process is known as page replacement and is a vital component in virtual memory management.

Why Need Page Replacement Algorithms?

To understand why we need page replacement algorithms, we first need to know about page faults. Let's see what is a page fault.

Page Fault: A Page Fault occurs when a program running in CPU tries to access a page that is in the address space of that program, but the requested page is currently not loaded into the main physical memory, the RAM of the system. Since the actual RAM is much less than the virtual memory the page faults occur. So whenever a page fault occurs, the Operating system has to replace an existing page in RAM with the newly requested page. In this scenario, page replacement algorithms help the Operating System in deciding which page to replace. The primary objective of all the page replacement algorithms is to minimize the number of page faults.

Page Replacement Algorithms in Operating Systems First In First Out

(FIFO)

The FIFO algorithm is the simplest of all the page replacement algorithms. In this, we maintain a queue of all the pages that are in the memory currently. The oldest page in the memory is at the front end of the queue and the most recent page is at the back or rear end of the queue.

Whenever a page fault occurs, the operating system looks at the front end of the queue to know the page to be replaced by the newly requested page. It also adds this newly requested page at the rear end and removes the oldest page from the front end of the queue.

Example: Consider the page reference string as 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames. Let's try to find the number of page faults:

Initially, all of the slots are empty so page faults occur at 3,1,2. Page faults = 3
When page 1 comes, it is in the memory so no page fault occurs. Page faults = 3
When page 6 comes, it is not present and a page fault occurs. Since there are no empty slots, we remove the front of the queue, i.e 3.

Page faults = 4
When page 5 comes, it is also not present, and hence a page fault occurs. The front of the queue i.e. 1 is removed.

Page faults = 5
When page 1 comes, it is not found in memory and again a page fault occurs. The front of the queue i.e. 2 is removed.

Page faults = 6
When page 3 comes, it is again not found in memory, a page fault occurs, and page 6 is removed being on top of the queue

Total page faults = 7

Belady's anomaly: Generally if we increase the number of frames in the memory, the number of page faults should decrease due to obvious reasons. Belady's anomaly refers to the phenomena where increasing the number of frames in memory, increases the page faults as well.

Advantages

- Simple to understand and implement.
- Does not cause more overhead.

Disadvantages

- Poor performance
- Doesn't use the frequency of the last used time and just simply replaces the oldest page.
- Suffers from Belady's anomaly.
- Optimal Page Replacement in OS

Optimal page replacement is the best page replacement algorithm as this algorithm results in the least number of page faults. In this algorithm, the pages are replaced with the ones that will not be used for the longest duration of time in the future. In simple terms, the pages that will be referred to farthest in the future are replaced in this algorithm.

Example:

Let's take the same page reference string 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames as we saw in FIFO. This also helps you understand how Optimal Page replacement works the best.

Initially, since all the slots are empty, pages 3, 1, 2 cause a page fault and take the empty slots.

Page faults = 3

When page 1 comes, it is in the memory and no page fault occurs. Page faults = 3

When page 6 comes, it is not in the memory, so a page fault occurs and 2 is removed as it is not going to be used again.

Page faults = 4

When page 5 comes, it is also not in the memory and causes a page fault. Similar to above 6 is removed as it is not going to be used again.

page faults = 5

When page 1 and page 3 come, they are in the memory so no page fault occurs. Total page faults = 5

Advantages

- Excellent efficiency.
- Less complexity.
- Easy to use and understand.
- Simple data structures can be used to implement.
- Used as the benchmark for other algorithms.

Disadvantages

- More time consuming
- Difficult for error handling
- Need future awareness of the programs, which is not possible every time

Least Recently Used (LRU) Page Replacement Algorithm

The least recently used page replacement algorithm keeps the track of usage of pages over a period of time. This algorithm works on the basis of the principle of locality of a reference which states that a program has a tendency to access the same set of memory locations repetitively over a short period of time. So pages that have been used heavily in the past are most likely to be used heavily in the future also.

In this algorithm, when a page fault occurs, then the page that has not been used for the longest duration of time is replaced by the newly requested page.

Example: Let's see the performance of the LRU on the same reference string of 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames:

Initially, since all the slots are empty, pages 3, 1, 2 cause a page fault and take the empty slots.

Page faults = 3

When page 1 comes, it is in the memory and no page fault occurs. Page faults = 3

When page 6 comes, it is not in the memory, so a page fault occurs and the least recently used page 3 is removed.

Page faults = 4

When page 5 comes, it again causes a page fault, and page 1 is removed as it is now the least recently used page.

Page faults = 5

When page 1 comes again, it is not in the memory, and hence page 2 is removed according to the LRU.

Page faults = 6

When page 3 comes, the page fault occurs again and this time page 6 is removed as the least recently used one.

Total page faults = 7

Now in the above example, the LRU causes the same page faults as the FIFO, but this may not always be the case as it will depend upon the series, the number of frames available in memory, etc. In fact, on most occasions, LRU is better than FIFO.

Advantages

- It is open for full analysis
- Doesn't suffer from Belady's anomaly
- Often more efficient than other algorithms

Disadvantages

- It requires additional data structures to be implemented
- More complex
- High hardware assistance is required.

Last In First Out (LIFO) Page Replacement Algorithm

This is the Last in First Out algorithm and works on LIFO principles. In this algorithm, the newest page is replaced by the requested page. Usually, this is done through a stack, where we maintain a stack of pages currently in the memory with the newest page being at the top. Whenever a page fault occurs, the page at the top of the stack is replaced.

Example: Let's see how the LIFO performs for our example string of 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames:

Initially, since all the slots are empty, page 3,1,2 causes a page fault and takes the empty slots.

Page faults = 3

When page 1 comes, it is in the memory and no page fault occurs. Page faults = 3

When page 6 comes, the page fault occurs and page 2 is removed as it is on the top of the stack and is the newest page.

Page faults = 4

When page 5 comes, it is not in the memory, which causes a page fault, and hence page 6 is removed being on top of the stack.

Page faults = 5

When page 1 and page 3 come, they are in memory already, hence no page fault occurs.

Total page faults = 5

As you may notice, this is the same number of page faults as the Optimal page replacement algorithm. So we can say that for this series of pages, this is the best algorithm that can be implemented without the prior knowledge of future references.

Advantages

- Simple to understand
- Easy to implement
- No overhead

Disadvantages

Does not consider Locality principle, hence may produce worst performance. The old pages may reside in memory forever even if they are not used.

Random Page Replacement in OS

This algorithm, as the name suggests, chooses any random page in the memory to be replaced by the requested page. This algorithm can behave like any of the algorithms based on the random page chosen to be replaced.

Example: Suppose we choose to replace the middle frame every time a page fault occurs. Let's see how our series of 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames perform with this algorithm:

Initially, since all the slots are empty, page 3,1,2 causes a page fault and takes the empty slots

Page faults = 3

When page 1 comes, it is in the memory and no page fault occurs. Page faults = 3

When page 6 comes, the page fault occurs, we replace the middle element i.e 1 is removed.

Page faults = 4

When page 5 comes, the page fault occurs again and middle element 6 is removed

Page faults = 5

When page 1 comes, there is again a page fault, and again the middle element 5 is removed

Page faults = 6

When page 3 comes, it is in memory, hence no page fault occurs. Total page faults = 6

As we can see, the performance is not the best, but it's also not the worst. The performance in the random replacement algorithm depends on the choice of the page chosen at random.

Advantages

- Easy to understand and implement
- No extra data structure needed to implement

- No overhead

Disadvantages

- Can not be analyzed, may produce different performances for the same series.
- Can suffer from Belady's anomaly.

Summary:

i. The objective of page replacement algorithms is to minimize the page faults
ii. FIFO page replacement algorithm replaces the oldest page in the memory
iii. Optimal page replacement algorithm replaces the page which will be referred farthest in the future
iv. LRU page replacement algorithm replaces the page that has not been used for the longest duration of time
v. LIFO page replacement algorithm replaces the newest page in memory
vi. Random page replacement algorithm replaces any page at random
vii. Optimal page replacement algorithm is considered to be the most effective algorithm but cannot be implemented in practical scenarios due to various limitations

❖ **CODE:**

❖ **Output:**

❖ **Conclusion:** In this practical, we conclude that the **LRU page replacement algorithm** helps manage memory efficiently by minimizing page faults and improving performance.

❖ **Discussion Questions:**

1. What is the Least Recently Used (LRU) page replacement algorithm? Ans: LRU is a page replacement algorithm that removes the least recently used page from memory when a new page needs to be loaded, ensuring better utilization of available frames.

2. How do you determine a page hit and a page fault?
Ans: A **page hit** occurs when a requested page is already in memory, while a

page fault occurs when the requested page is not in memory and must be loaded from disk.

3. What is the formula to calculate page hit and page fault percentage? Ans:

- **Page Hit Percentage** = (Page Hits / Total Page Requests) × 100
- **Page Fault Percentage** = (Page Faults / Total Page Requests) × 100

4. Why is LRU preferred over FIFO for page replacement?
Ans: LRU is preferred over FIFO because it removes the page that has not been used for the longest time, leading to better performance, whereas FIFO may remove frequently used pages.

5. What is the worst-case scenario for LRU page replacement?
Ans: The worst case occurs when all incoming pages are unique and do not repeat within the limited frames, leading to the **maximum number of page faults**.

❖ **References:**
https://www.scaler.com/topics/operating-system/page-replacement-algorithm/ https://chatgpt.com/

Date:____/____/2026

Signature Course Coordinator B.Tech CSE(AIML)
Sem: 4 / 2025-26

S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 09

Aim: A system has RAM which can store four pages simultaneously to satisfy page requests. Write a Program to calculate percentage page hit and page fault if the system uses Optimal page replacement technique for the page references entered by the user i.e. 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (No of Frames=3).

Name:

USN:

Semester / Year: Academic

Session: Date of

Performance: Date of

Submission:

❖ **Aim:** A system has RAM which can store four pages simultaneously to satisfy page requests. Write a Program to calculate percentage page hit and page fault if the system uses Least recently used page replacement technique for the page references entered by the user i.e. 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (No of Frames=3).

❖ **Objectives:**

1. Implement the Least Recently Used (LRU) page replacement algorithm for memory management.
2. Calculate the number of page hits and page faults for a given page reference sequence.
3. Determine the percentage of page hits and page faults based on the system's frame capacity.

❖ **Requirements:**

✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

✓ **Software Requirements:**

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

❖ **Theory:**

In an operating system, page replacement refers to a scenario in which a page from the main memory should be replaced by a page from the secondary memory. Page replacement occurs due to page faults. The various page replacement algorithms like FIFO, Optimal page replacement, LRU, LIFO, and Random page replacement help the operating system decide which page to replace.

What is Page Replacement in Operating Systems?

Page replacement is needed in the operating systems that use virtual memory using Demand Paging. As we know in Demand paging, only a set of pages of a process is loaded into the memory. This is done so that we can have more processes in the memory at the same time.

When a page that is residing in virtual memory is requested by a process for its execution, the Operating System needs to decide which page will be replaced by this requested page. This process is known as page replacement and is a vital component in virtual memory management.

Why Need Page Replacement Algorithms?

To understand why we need page replacement algorithms, we first need to know about page faults. Let's see what is a page fault.

Page Fault: A Page Fault occurs when a program running in CPU tries to access a page that is in the address space of that program, but the requested page is currently not loaded into the main physical memory, the RAM of the system. Since the actual RAM is much less than the virtual memory the page faults occur. So whenever a page fault occurs, the Operating system has to replace an existing page in RAM with the newly requested page. In this scenario, page replacement algorithms help the Operating System in deciding which page to replace. The primary objective of all the page replacement algorithms is to minimize the number of page faults.

Page Replacement Algorithms in Operating Systems First In First Out

(FIFO)

The FIFO algorithm is the simplest of all the page replacement algorithms. In this, we maintain a queue of all the pages that are in the memory currently. The oldest page in the memory is at the front end of the queue and the most recent page is at the back or rear end of the queue.

Whenever a page fault occurs, the operating system looks at the front end of the queue to know the page to be replaced by the newly requested page. It also adds this newly requested page at the rear end and removes the oldest page from the front end of the queue.

Example: Consider the page reference string as 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames. Let's try to find the number of page faults:

Initially, all of the slots are empty so page faults occur at 3,1,2. Page faults = 3
When page 1 comes, it is in the memory so no page fault occurs. Page faults = 3
When page 6 comes, it is not present and a page fault occurs. Since there are no empty slots, we remove the front of the queue, i.e 3.

Page faults = 4
When page 5 comes, it is also not present, and hence a page fault occurs. The front of the queue i.e. 1 is removed.

Page faults = 5
When page 1 comes, it is not found in memory and again a page fault occurs. The front of the queue i.e. 2 is removed.

Page faults = 6
When page 3 comes, it is again not found in memory, a page fault occurs, and page 6 is removed being on top of the queue

Total page faults = 7

Belady's anomaly: Generally if we increase the number of frames in the memory, the number of page faults should decrease due to obvious reasons. Belady's anomaly refers to the phenomena where increasing the number of frames in memory, increases the page faults as well.

Advantages

- Simple to understand and implement.
- Does not cause more overhead.

Disadvantages

- Poor performance
- Doesn't use the frequency of the last used time and just simply replaces the oldest page.
- Suffers from Belady's anomaly.
- Optimal Page Replacement in OS

Optimal page replacement is the best page replacement algorithm as this algorithm results in the least number of page faults. In this algorithm, the pages are replaced with the ones that will not be used for the longest duration of time in the future. In simple terms, the pages that will be referred to farthest in the future are replaced in this algorithm.

Example:

Let's take the same page reference string 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames as we saw in FIFO. This also helps you understand how Optimal Page replacement works the best.

Initially, since all the slots are empty, pages 3, 1, 2 cause a page fault and take the empty slots.

Page faults = 3

When page 1 comes, it is in the memory and no page fault occurs. Page faults = 3

When page 6 comes, it is not in the memory, so a page fault occurs and 2 is removed as it is not going to be used again.

Page faults = 4

When page 5 comes, it is also not in the memory and causes a page fault. Similar to above 6 is removed as it is not going to be used again.

page faults = 5

When page 1 and page 3 come, they are in the memory so no page fault occurs. Total page faults = 5

Advantages

- Excellent efficiency.
- Less complexity.
- Easy to use and understand.
- Simple data structures can be used to implement.
- Used as the benchmark for other algorithms.

Disadvantages

- More time consuming
- Difficult for error handling
- Need future awareness of the programs, which is not possible every time

Least Recently Used (LRU) Page Replacement Algorithm

The least recently used page replacement algorithm keeps the track of usage of pages over a period of time. This algorithm works on the basis of the principle of locality of a reference which states that a program has a tendency to access the same set of memory locations repetitively over a short period of time. So pages that have been used heavily in the past are most likely to be used heavily in the future also.

In this algorithm, when a page fault occurs, then the page that has not been used for the longest duration of time is replaced by the newly requested page.

Example: Let's see the performance of the LRU on the same reference string of 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames:

Initially, since all the slots are empty, pages 3, 1, 2 cause a page fault and take the empty slots.

Page faults = 3

When page 1 comes, it is in the memory and no page fault occurs. Page faults = 3

When page 6 comes, it is not in the memory, so a page fault occurs and the least recently used page 3 is removed.

Page faults = 4

When page 5 comes, it again causes a page fault, and page 1 is removed as it is now the least recently used page.

Page faults = 5

When page 1 comes again, it is not in the memory, and hence page 2 is removed according to the LRU.

Page faults = 6

When page 3 comes, the page fault occurs again and this time page 6 is removed as the least recently used one.

Total page faults = 7

Now in the above example, the LRU causes the same page faults as the FIFO, but this may not always be the case as it will depend upon the series, the number of frames available in memory, etc. In fact, on most occasions, LRU is better than FIFO.

Advantages

- It is open for full analysis
- Doesn't suffer from Belady's anomaly
- Often more efficient than other algorithms

Disadvantages

- It requires additional data structures to be implemented
- More complex
- High hardware assistance is required.

Last In First Out (LIFO) Page Replacement Algorithm

This is the Last in First Out algorithm and works on LIFO principles. In this algorithm, the newest page is replaced by the requested page. Usually, this is done through a stack, where we maintain a stack of pages currently in the memory with the newest page being at the top. Whenever a page fault occurs, the page at the top of the stack is replaced.

Example: Let's see how the LIFO performs for our example string of 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames:

Initially, since all the slots are empty, page 3,1,2 causes a page fault and takes the empty slots.

Page faults = 3

When page 1 comes, it is in the memory and no page fault occurs. Page faults = 3

When page 6 comes, the page fault occurs and page 2 is removed as it is on the top of the stack and is the newest page.

Page faults = 4

When page 5 comes, it is not in the memory, which causes a page fault, and hence page 6 is removed being on top of the stack.

Page faults = 5

When page 1 and page 3 come, they are in memory already, hence no page fault occurs.

Total page faults = 5

As you may notice, this is the same number of page faults as the Optimal page replacement algorithm. So we can say that for this series of pages, this is the best algorithm that can be implemented without the prior knowledge of future references.

Advantages

- Simple to understand
- Easy to implement
- No overhead

Disadvantages

Does not consider Locality principle, hence may produce worst performance. The old pages may reside in memory forever even if they are not used.

Random Page Replacement in OS

This algorithm, as the name suggests, chooses any random page in the memory to be replaced by the requested page. This algorithm can behave like any of the algorithms based on the random page chosen to be replaced.

Example: Suppose we choose to replace the middle frame every time a page fault occurs. Let's see how our series of 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames perform with this algorithm:

Initially, since all the slots are empty, page 3,1,2 causes a page fault and takes the empty slots

Page faults = 3

When page 1 comes, it is in the memory and no page fault occurs. Page faults = 3

When page 6 comes, the page fault occurs, we replace the middle element i.e 1 is removed.

Page faults = 4

When page 5 comes, the page fault occurs again and middle element 6 is removed

Page faults = 5

When page 1 comes, there is again a page fault, and again the middle element 5 is removed

Page faults = 6

When page 3 comes, it is in memory, hence no page fault occurs. Total page faults = 6

As we can see, the performance is not the best, but it's also not the worst. The performance in the random replacement algorithm depends on the choice of the page chosen at random.

Advantages

- Easy to understand and implement
- No extra data structure needed to implement

- No overhead

Disadvantages

- Can not be analyzed, may produce different performances for the same series.
- Can suffer from Belady's anomaly.

Summary:

viii. The objective of page replacement algorithms is to minimize the page faults
ix. FIFO page replacement algorithm replaces the oldest page in the memory
x. Optimal page replacement algorithm replaces the page which will be referred farthest in the future
xi. LRU page replacement algorithm replaces the page that has not been used for the longest duration of time
xii. LIFO page replacement algorithm replaces the newest page in memory
xiii. Random page replacement algorithm replaces any page at random
xiv. Optimal page replacement algorithm is considered to be the most effective algorithm but cannot be implemented in practical scenarios due to various limitations

❖ **CODE:**

❖ **Output:**

❖ **Conclusion:** In this practical, we conclude that the **LRU page replacement algorithm** helps manage memory efficiently by minimizing page faults and improving performance.

❖ **Discussion Questions:**

1. What is the Optimal page replacement algorithm?

2. Ans:

3. How do you determine a page hit and a page fault?
Ans:

4. What is the formula to calculate page hit and page fault percentage? Ans:

5. Why is Optimal preferred over FIFO for page replacement?
Ans:

6. What is the worst-case scenario for Optimal page replacement?
Ans:

❖ **References:**
https://www.scaler.com/topics/operating-system/page-replacement-algorithm/ https://chatgpt.com/

Date:____/____/2026

Signature Course Coordinator B.Tech CSE(AIML)
Sem: 4 / 2025-26

S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 10

Aim: Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The

SSTF scheduling algorithm is used. The head is initially at cylinder number 53. The cylinders are numbered from 0 to 199. Write a Program to calculate the total head movement (in number of cylinders) incurred while servicing these requests.

Name:

USN:

Semester / Year: Academic

Session: Date of

Performance: Date of

Submission:

- **Aim:** Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The SSTF scheduling algorithm is used. The head is initially at cylinder number 53. The cylinders are numbered from 0 to 199. Write a Program to calculate the total head movement (in number of cylinders) incurred while servicing these requests.
- **Objectives:**
  a) Implement the **Shortest Seek Time First (SSTF)** disk scheduling algorithm to optimize disk head movement.
  b) Calculate the total **head movement** required to service all I/O requests efficiently.
  c) Analyze the performance of SSTF by minimizing the seek time compared to other scheduling algorithms.
- **Requirements:**
  - ✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

  - ✓ **Software Requirements:**

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

- **Theory:**

Disk scheduling algorithms are crucial in managing how data is read from and written to a computer's hard disk. These algorithms help determine the order in which disk read and write requests are processed, significantly impacting the speed and efficiency of data access. Common disk scheduling methods include First-Come, First-Served (FCFS), Shortest Seek Time First (SSTF), SCAN, C- SCAN, LOOK, and C-LOOK. By understanding and implementing these algorithms, we can optimize system performance and ensure faster data retrieval.

- Disk scheduling is a technique operating systems use to manage the order in which disk I/O (input/output) requests are processed.
- Disk scheduling is also known as I/O Scheduling.
- The main goals of disk scheduling are to optimize the performance of disk operations, reduce the time it takes to access data and improve overall system efficiency.

In this article, we will explore the different types of disk scheduling algorithms and their functions. By understanding and implementing these algorithms, we can optimize system performance and ensure faster data retrieval.

Importance of Disk Scheduling in Operating System

- Multiple I/O requests may arrive by different processes and only one I/O request can be served at a time by the disk controller. Thus other I/O requests need to wait in the waiting queue and need to be scheduled.
- Two or more requests may be far from each other so this can result in greater disk arm movement.
- Hard drives are one of the slowest parts of the computer system and thus need to be accessed in an efficient manner.

Key Terms Associated with Disk Scheduling

- **Seek Time:** Seek time is the time taken to locate the disk arm to a specified track where the data is to be read or written. So the disk scheduling algorithm that gives a minimum average seek time is better.
- **Rotational Latency:** Rotational Latency is the time taken by the desired sector of the disk to rotate into a position so that it can access the read/write heads. So the disk scheduling algorithm that gives minimum rotational latency is better.
- **Transfer Time:** Transfer time is the time to transfer the data. It depends on the rotating speed of the disk and the number of bytes to be transferred.
- **Disk Access Time:**
  *Disk Access Time = Seek Time + Rotational Latency + Transfer Time Total Seek Time = Total head Movement * Seek Time*

- **Disk Response Time:** Response Time is the average time spent by a request waiting to perform its I/O operation. The average *Response time* is the response time of all requests. *Variance Response Time* is the measure of how individual requests are serviced with respect to average response time. So the disk scheduling algorithm that gives minimum variance response time is better.
  Goal of Disk Scheduling Algorithms
- Minimize Seek Time
- Maximize Throughput
- Minimize Latency
- Fairness

- Efficiency in Resource Utilization Disk Scheduling Algorithms

  There are several Disk Several Algorithms. We will discuss in detail each one of them.
- **FCFS (First Come First Serve)**
- **SSTF (Shortest Seek Time First)**
- **SCAN**
- **C-SCAN**
- **LOOK**
- **C-LOOK**
- **RSS (Random Scheduling)**
- **LIFO (Last-In First-Out)**
- **N-STEP SCAN**
- **F-SCAN**

1. FCFS (First Come First Serve)

FCFS is the simplest of all Disk Scheduling Algorithms. In FCFS, the requests are addressed in the order they arrive in the disk queue. Let us understand this with the help of an example.

Example:

Suppose the order of request is- (82,170,43,140,24,16,190) And current position of Read/Write head is: 50

So, total overhead movement (total distance covered by the disk arm) = (82-50)+(170-82)+(170-43)+(140-43)+(140-24)+(24-16)+(190-16) =642

Advantages of FCFS

Here are some of the advantages of First Come First Serve.

- Every request gets a fair chance
- No indefinite postponement

Disadvantages of FCFS

Here are some of the disadvantages of First Come First Serve.

- Does not try to optimize seek time
- May not provide the best possible service

2. SSTF (Shortest Seek Time First)

In SSTF (Shortest Seek Time First), requests having the shortest seek time are executed first. So, the seek time of every request is calculated in advance in the queue and then they are scheduled according to their calculated seek time. As a result, the request near the disk arm will get executed first. SSTF is certainly an improvement over FCFS as it decreases the average response time and increases the throughput of the system. Let us understand this with the help of an example. **Example:**

Suppose the order of request is- (82,170,43,140,24,16,190) And current position of Read/Write head is: 50

So,

total overhead movement (total distance covered by the disk arm) =

(50-43)+(43-24)+(24-16)+(82-16)+(140-82)+(170-140)+(190-170) =208

Advantages of Shortest Seek Time First

Here are some of the advantages of Shortest Seek Time First.

- The average Response Time decreases
- Throughput increases

Disadvantages of Shortest Seek Time First

Here are some of the disadvantages of Shortest Seek Time First.

- Overhead to calculate seek time in advance
- Can cause Starvation for a request if it has a higher seek time as compared to incoming requests
- The high variance of response time as SSTF favors only some requests

3. SCAN

In the SCAN algorithm the disk arm moves in a particular direction and services the requests coming in its path and after reaching the end of the disk, it reverses its direction and again services the request arriving in its path. So, this algorithm works as an elevator and is hence also known as an **elevator algorithm.** As a result, the requests at the midrange are serviced more and those arriving behind the disk arm will have to wait.

Example:

Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move **"towards the larger value".**
Therefore, the total overhead movement (total distance covered by the disk arm) is calculated as

= (199-50) + (199-16) = 332

Advantages of SCAN Algorithm

Here are some of the advantages of the SCAN Algorithm.

- High throughput
- Low variance of response time
- Average response time

Disadvantages of SCAN Algorithm

Here are some of the disadvantages of the SCAN Algorithm.

- Long waiting time for requests for locations just visited by disk arm

4. C-SCAN

In the SCAN algorithm, the disk arm again scans the path that has been scanned, after reversing its direction. So, it may be possible that too many requests are waiting at the other end or there may be zero or few requests pending at the scanned area.
These situations are avoided in the *CSCAN* algorithm in which the disk arm instead of reversing its direction goes to the other end of the disk and starts servicing the requests from there. So, the disk arm moves in a circular fashion and this algorithm is also similar to the SCAN algorithm hence it is known as C-SCAN (Circular SCAN).
Example:

Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move **"towards the larger value".**
So, the total overhead movement (total distance covered by the disk arm) is calculated as:

=(199-50) + (199-0) + (43-0) = 391

Advantages of C-SCAN Algorithm

Here are some of the advantages of C-SCAN.

- Provides more uniform wait time compared to SCAN.

5. LOOK

LOOK Algorithm is similar to the SCAN disk scheduling algorithm except for the difference that the disk arm in spite of going to the end of the disk goes only to the last request to be serviced in front of the head and then reverses its direction from there only. Thus it prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.

Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move **"towards the larger value".**

So, the total overhead movement (total distance covered by the disk arm) is calculated as:

= (190-50) + (190-16) = 314

6. C-LOOK

As LOOK is similar to the SCAN algorithm, in a similar way, C-LOOK is similar to the CSCAN disk scheduling algorithm. In CLOOK, the disk arm in spite of going to the end goes only to the last request to be serviced in front of the head and then from there goes to the other end's last request. Thus, it also prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.

Example:

1. Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move **"towards the larger value"**

So, the total overhead movement (total distance covered by the disk arm) is calculated as

= (190-50) + (190-16) + (43-16) = 341

7. RSS (Random Scheduling)

It stands for Random Scheduling and just like its name it is natural. It is used in situations where scheduling involves random attributes such as random processing time, random due dates, random weights, and stochastic machine breakdowns this algorithm sits perfectly. Which is why it is usually used for analysis and simulation.

8. LIFO (Last-In First-Out)

In LIFO (Last In, First Out) algorithm, the newest jobs are serviced before the existing ones i.e. in order of requests that get serviced the job that is newest or last entered is serviced first, and then the rest in the same order.

Advantages of LIFO (Last-In First-Out)

Here are some of the advantages of the Last In First Out Algorithm.

- Maximizes locality and resource utilization
- Can seem a little unfair to other requests and if new requests keep coming in, it cause starvation to the old and existing ones.

9. N-STEP SCAN

It is also known as the N-STEP LOOK algorithm. In this, a buffer is created for N requests. All requests belonging to a buffer will be serviced in one go. Also once the buffer is full no new requests are kept in this buffer and are sent to another one. Now, when these N requests are serviced, the time comes for another top N request and this way all get requests to get a guaranteed service **Advantages of N-STEP SCAN**

Here are some of the advantages of the N-Step Algorithm.

- It eliminates the starvation of requests completely

10. F-SCAN

This algorithm uses two sub-queues. During the scan, all requests in the first queue are serviced and the new incoming requests are added to the second queue. All new requests are kept on halt until the existing requests in the first queue are serviced.

Advantages of F-SCAN

Here are some of the advantages of the F-SCAN Algorithm.

- F-SCAN along with N-Step-SCAN prevents "arm stickiness" (phenomena in I/O scheduling where the scheduling algorithm continues to service requests at or near the current sector and thus prevents any seeking) Each algorithm is unique in its own way. Overall Performance depends on the number and type of requests.

Note: Average Rotational latency is generally taken as 1/2(Rotational latency)

❖ **CODE:**

❖ **Output:**

❖ **Conclusion**: In this practical, we conclude that the Shortest Seek Time First (SSTF) scheduling algorithm efficiently reduces total head movement by servicing the nearest cylinder request first. This approach minimizes seek time, improving disk performance compared to other scheduling methods.

❖ **Discussion Questions:**

1. What is the Shortest Seek Time First (SSTF) disk scheduling algorithm? Ans: SSTF is a disk scheduling algorithm that selects the I/O request closest to the current head position, minimizing seek time and improving performance.

2. **How does SSTF reduce the total head movement?**
   Ans: SSTF services requests in order of proximity to the current head position, reducing unnecessary long movements and optimizing disk access time.

3. **What is the drawback of SSTF scheduling?**
   Ans: SSTF can lead to **starvation**, where requests for distant cylinders may experience indefinite delays due to the continuous selection of nearby requests.

4. **How is total head movement calculated in SSTF?**
   Ans: Total head movement is calculated as the sum of the absolute differences between consecutive requests in the order they are serviced.

5. **How does SSTF compare to the FCFS scheduling algorithm?**

Ans: SSTF reduces seek time compared to **First-Come, First-Served (FCFS)**, which processes requests in arrival order but may result in larger head movements and inefficiency.

❖ **References:**

https://www.geeksforgeeks.org/disk-scheduling-algorithms/ https://chatgpt.com/

Date:_____/_____/2026

Signature Course
Coordinator B.Tech
CSE(AIML)
Sem: 4 / 2025-26

S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 11
Postlab (OPEN ENDED PRACTICAL)

Aim: Implementing a Virtual Memory System with Demand Paging where pages are loaded into memory only when needed, optimizing memory usage and minimizing page faults.

Name:

USN:

Semester / Year: Academic

Session: Date of

Performance: Date of

Submission:

❖ **Aim:** Implementing a Virtual Memory System with Demand Paging where pages are loaded into memory only when needed, optimizing memory usage and minimizing page faults.

❖ **Objectives:**

i. Implement demand paging to load pages only when required.
ii. Optimize memory utilization and reduce page faults.
iii. Simulate page replacement algorithms for efficient management.

❖ **Requirements:**

✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

✓ **Software Requirements:**

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

❖ **Theory:**

What is Demand Paging?

Demand paging is a technique used in virtual memory systems where pages enter main memory only when requested or needed by the CPU. In demand paging, the operating system loads only the necessary pages of a program into memory at runtime, instead of loading the entire program into memory at the start. A page fault occurred when the program needed to access a page that is not currently in memory.

The operating system then loads the required pages from the disk into memory and updates the page tables accordingly. This process is transparent to the running program and it continues to run as if the page had always been in memory.

Pure Demand Paging

Pure demand paging is a specific implementation of demand paging. The operating system only loads pages into memory when the program needs them. In on-demand paging only, no pages are initially loaded into memory when the program starts, and all pages are initially marked as being on disk.

Operating systems that use pure demand paging as a memory management strategy do so without preloading any pages into physical memory prior to the commencement of a task. Demand paging loads a process's whole address space

into memory one step at a time, bringing just the parts of the process that are actively being used into memory from disc as needed.
It is useful for executing huge programs that might not fit totally in memory or for computers with limited physical memory. If the program accesses a lot of pages that are not in memory right now, it could also result in a rise in page faults and possible performance overhead. Operating systems frequently use caching techniques and improve page replacement algorithms to lessen the negative effects of page faults on system performance as a whole.

Working Process of Demand Paging
Let us understand this with the help of an example. Suppose we want to run a process P which have four pages P0, P1, P2, and P3. Currently, in the page table, we have pages P1 and P3.

The operating system's demand paging mechanism follows a few steps in its operation.

- **Program Execution:** Upon launching a program, the operating system allocates a certain amount of memory to the program and establishes a process for it.
- **Creating Page Tables:** To keep track of which program pages are currently in memory and which are on disk, the operating system makes page tables for each process.
- **Handling Page Fault:** When a program tries to access a page that isn't in memory at the moment, a page fault happens. In order to determine whether the necessary page is on disk, the operating system pauses the application and consults the page tables.
- **Page Fetch:** The operating system loads the necessary page into memory by retrieving it from the disk if it is there.
- The page's new location in memory is then reflected in the page table.

- **Resuming The Program:** The operating system picks up where it left off when the necessary pages are loaded into memory.
- **Page Replacement:** If there is not enough free memory to hold all the pages a program needs, the operating system may need to replace one or more pages currently in memory with pages currently in memory. on the disk. The page replacement algorithm used by the operating system determines which pages are selected for replacement.
- **Page Cleanup:** When a process terminates, the operating system frees the memory allocated to the process and cleans up the corresponding entries in the page tables.

Advantages of Demand Paging

So in the Demand Paging technique, there are some benefits that provide efficiency of the operating system.

- **Efficient use of physical memory**: Query paging allows for more efficient use because only the necessary pages are loaded into memory at any given time.
- **Support for larger programs:** Programs can be larger than the physical memory available on the system because only the necessary pages will be loaded into memory.
- **Faster program start:** Because only part of a program is initially loaded into memory, programs can start faster than if the entire program were loaded at once.
- **Reduce memory usage:** Query paging can help reduce the amount of memory a program needs, which can improve system performance by reducing the amount of disk I/O required.

Disadvantages of Demand Paging

- **Page Fault Overload:** The process of swapping pages between memory and disk can cause a performance overhead, especially if the program frequently accesses pages that are not currently in memory.
- **Degraded Performance:** If a program frequently accesses pages that are not currently in memory, the system spends a lot of time swapping out pages, which degrades performance.
- **Fragmentation:** Query paging can cause physical memory fragmentation, degrading system performance over time.
- **Complexity:** Implementing query paging in an operating system can be complex, requiring complex algorithms and data structures to manage page tables and swap space.

What is the Impact of Demand Paging in Virtual Memory Management?

With demand paging, the operating system swaps memory pages between the main memory and secondary storage based on need. When a program needs a page not currently in memory, the operating system retrieves it from secondary storage, a process called a page fault.

Demand paging significantly impacts virtual memory management by allowing the operating system to use virtual memory efficiently, improving overall system performance. Its main advantage is reducing the physical memory required, enabling more applications to run at once and allowing larger programs to run.
However, demand paging has some drawbacks. The page fault mechanism can delay program execution because the operating system must retrieve pages from secondary storage. This delay can be minimized by optimizing the page replacement algorithm.

Virtual Memory vs Physical Memory

Feature	**Virtual Memory**	**Physical Memory (RAM)**
Definition	An abstraction that extends the available memory by using disk storage	The actual hardware (RAM) that stores data and instructions currently being used by the CPU
Location	On the hard drive or SSD	On the computer's motherboard
Speed	Slower (due to disk I/O operations)	Faster (accessed directly by the CPU)
Capacity	Larger, limited by disk space	Smaller, limited by the amount of RAM installed
Cost	Lower (cost of additional disk storage)	Higher (cost of RAM modules)
Data Access	Indirect (via paging and swapping)	Direct (CPU can access data directly)
Volatility	Non-volatile (data persists on disk)	Volatile (data is lost when power is off)

- **CODE:**

- **Output:**

- **Conclusion**: In this practical, we conclude that demand paging efficiently manages memory by loading pages only when needed, reducing page faults and optimizing system performance.

- **Discussion Questions:**

1. What is Demand Paging in Virtual Memory?

Ans: Demand paging is a memory management technique where pages are loaded into RAM only when they are required, reducing unnecessary memory usage.

2. What causes a page fault in Demand Paging?

Ans: A page fault occurs when a program tries to access a page that is not currently in RAM, triggering the operating system to load it from secondary storage.

3. What are the advantages of Demand Paging?

Ans: It reduces memory wastage, improves multitasking efficiency, and allows programs to run with less RAM.

4. Which page replacement algorithms are commonly used in Demand Paging?

Ans: Common algorithms include FIFO (First-In-First-Out), LRU (Least Recently Used), and Optimal Page Replacement.

5. How does Demand Paging differ from Pre-paging?

Ans: Demand paging loads pages only when required, while pre-paging loads multiple pages in advance to reduce future page faults.

- **References:**

https://www.geeksforgeeks.org/disk-scheduling-algorithms/ https://chatgpt.com/

Date:_____/_____/2026

Signature Course Coordinator B.Tech CSE(AIML)
Sem: 4 / 2025-26